Real-Time Network Intrusion Detection System (NIDS) using Packet Inspection and Web-Based Alerting

---

MINI PROJECT REPORT

Submitted in partial fulfilment of the requirement for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

Submitted to

DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, UTTAR PRADESH, LUCKNOW

Submitted by

Name: Rohan Pandey
Roll No.: 2501720100117
Semester & Section: Semester III, Section 2D

Under the Supervision of

Mr. Vikas
Assistant Professor

Department of Computer Science & Engineering & Allied Branches

Lloyd Institute of Management and Technology

Plot No. 3, Knowledge Park II, Greater Noida, Uttar Pradesh 201306

Session: 2026–27

CERTIFICATE

This is to certify that the Mini Project Report entitled "*Real-Time Network Intrusion Detection System (NIDS) using Packet Inspection and Web-Based Alerting*" is a bonafide record of the work carried out by **Rohan Pandey** (Roll No. 2501720100117), a student of Semester III, Section 2D, Computer Science & Engineering & Allied Branches, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering during the academic session 2026-27.

The work embodied in this report has not been submitted elsewhere for the award of any other degree or diploma. To the best of my knowledge, the report fulfils the prescribed structure and formatting requirements of the Mini Project guidelines.

_____

Mr. Vikas (Project Guide)

Assistant Professor, Department of Computer Science & Engineering & Allied Branches

Date: _____

Place: Greater Noida

DECLARATION

I hereby declare that the Mini Project Report entitled "*Real-Time Network Intrusion Detection System (NIDS) using Packet Inspection and Web-Based Alerting*", submitted to the Department of Computer Science & Engineering & Allied Branches, Lloyd Institute of Management and Technology, in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science & Engineering, is a record of original work carried out by me under the guidance of Mr. Vikas, Assistant Professor.

I further declare that the work reported here has not been copied from any other source, and that all references, data and material taken from other sources have been duly acknowledged in the report. The similarity index of this report, as checked using the institution's tool, is below the permitted limit of 20% (excluding references, code and standard boilerplate text).

Signature: _____

Name: **Rohan Pandey**

Roll No.: 2501720100117

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Mr. Vikas**, Assistant Professor, Department of Computer Science & Engineering & Allied Branches, for the valuable guidance, constant encouragement and constructive suggestions throughout the development of this mini project.

I am thankful to the Project Coordinator and the Head of the Department of Computer Science and Engineering for providing the necessary facilities and a supportive academic environment. Their support made it possible to carry out this work smoothly.

Finally, I thank all the faculty members, the laboratory staff, my family and my friends for their continuous support and motivation during the course of this work.

Rohan Pandey

Roll No. 2501720100117

ABSTRACT

Small networks such as homes, student hostels and college laboratories are rarely monitored, so an attacker can probe or disrupt them without anyone noticing. This project presents a lightweight, real-time Network Intrusion Detection System (NIDS) that captures live network traffic, recognises three common attack patterns and reports them on a web dashboard within about a second. A sensor written in Python uses the Scapy library over the Npcap driver to capture TCP, UDP and ARP packets from the wireless and loopback interfaces. A rule engine, written from first principles rather than using machine learning, inspects every packet: it flags a port scan when one address contacts more than fifteen distinct ports within a five-second window, a SYN flood when a host receives many half-open connections, and ARP spoofing when a trusted gateway address appears with a different hardware address. Each alert is sent as a JSON message over HTTP to a Node.js and Express server, which logs it and pushes it to the browser through Socket.IO, so the dashboard updates without being refreshed. The complete system is about two hundred lines of code and runs on an ordinary laptop. It was validated with ten automated unit tests and with self-generated attack traffic: all three attacks were detected correctly, duplicate alerts were suppressed by a cool-down, and the detection latency was a few milliseconds. The design favours transparency, because every alert can be traced to a specific counter and threshold, which makes the system easy to explain and extend.

TABLE OF CONTENTS

Certificate ..... **i**
Declaration ..... **ii**
Acknowledgement ..... **iii**
Abstract ..... **iv**
List of Figures ..... **vii**
List of Tables ..... **viii**
List of Abbreviations ..... **ix**
CHAPTER 1: INTRODUCTION **1**
1.1 Background 1
1.2 Motivation 2
1.3 Problem Statement 2
1.4 Objectives 2
1.5 Scope of the Project 3
1.6 Organisation of the Report 3
CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM **4**
2.1 Intrusion Detection Systems 4
2.2 Existing Tools 5
2.3 Attack Detection Techniques 5
2.4 Rule-Based Detection versus Machine Learning 6
2.5 Comparison and Identified Gap 6
2.6 Summary 7
CHAPTER 3: SYSTEM ANALYSIS AND DESIGN **8**
3.1 Requirement Analysis 8
3.2 Feasibility Study 9
3.3 System Architecture 9
3.4 Data Flow Diagrams 10
3.5 Use-Case Diagram 11
3.6 Sequence Diagram 12
3.7 Component Diagram 13
3.8 Deployment Diagram 14
3.9 Data Design 15
3.10 Design Summary 16
CHAPTER 4: IMPLEMENTATION **18**
4.1 Development Environment 18
4.2 Configuration Module 18
4.3 Packet Capture and Parsing 19
4.4 Detection Rules 20

4.5 Concurrency Model 22
4.6 Event-Pipeline Server 23
4.7 Real-Time Dashboard 24
4.8 Starting and Running the System 25
4.9 Statistics Reporting and Start-up 26
4.10 Security Considerations 27
CHAPTER 5: TESTING 28
5.1 Testing Strategy 28
5.2 Unit Testing 28
5.3 Integration Testing 29
5.4 Sample Input and Output 29
5.5 Reproducing the Tests 30
CHAPTER 6: RESULTS AND DISCUSSION 31
6.1 The Running System 31
6.2 Performance Measurements 33
6.3 Discussion 34
6.4 Evaluation against the Objectives 35
6.5 Summary of Strengths and Limitations 35
6.6 Comparison with Existing Tools in Practice 35
CHAPTER 7: CONCLUSION AND FUTURE SCOPE 37
7.1 Conclusion 37
7.2 Future Scope 38
REFERENCES 39
APPENDIX A: SOURCE CODE 41

LIST OF FIGURES

Fig. 2.1 Taxonomy of intrusion detection systems and the place of this project 4
Fig. 3.1 Overall architecture of the real-time NIDS 10
Fig. 3.2 Level-0 data flow diagram (context diagram) 10
Fig. 3.3 Level-1 data flow diagram 11
Fig. 3.4 Use-case diagram of the system 12
Fig. 3.5 Sequence diagram from packet arrival to on-screen alert 13
Fig. 3.6 Component diagram of the three subsystems 14
Fig. 3.7 Deployment diagram of the prototype 15
Fig. 3.8 Layers of a captured frame and the fields the NIDS reads 16
Fig. 4.1 Flowchart of packet processing and port-scan detection 21
Fig. 4.2 The three threads of the sensor and the queue that connects them 23
Fig. 6.1 Dashboard under normal traffic, with live counts and chart 31
Fig. 6.2 Dashboard showing a detected port scan (medium severity) 32
Fig. 6.3 Dashboard showing a detected SYN flood (high severity) 32
Fig. 6.4 Alert history preserved after a page reload 33

LIST OF TABLES

Table 2.1 Comparison of existing tools with the proposed system 6
Table 3.1 Functional requirements 8
Table 3.2 Non-functional requirements 8
Table 3.3 Fields of an alert record 15
Table 3.4 Packet fields read by the detector 16
Table 3.5 Data dictionary of the detection module 16
Table 4.1 Development environment 18
Table 4.2 Configuration parameters 19
Table 4.3 Detection thresholds 22
Table 4.4 HTTP endpoints exposed by the server 23
Table 5.1 Unit test cases and results 28
Table 5.2 Integration test results 29
Table 6.1 Performance measurements 34
Table 6.2 Evaluation against the stated objectives 35
Table 6.3 Strengths and limitations of the system 35

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ACK	Acknowledgement (TCP flag)
API	Application Programming Interface
ARP	Address Resolution Protocol
BPF	Berkeley Packet Filter
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DoS	Denial of Service
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IDS	Intrusion Detection System
IP	Internet Protocol
IPS	Intrusion Prevention System
JSON	JavaScript Object Notation
MAC	Media Access Control (address)
NIDS	Network-based Intrusion Detection System
RST	Reset (TCP flag)
SYN	Synchronise (TCP flag)
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
UML	Unified Modeling Language

CHAPTER 1: INTRODUCTION

1.1 Background

Almost every activity in a modern home, hostel or college laboratory now depends on a small network. The same wireless link that delivers web pages, video calls and software updates also carries attacks. Before a serious attack, an intruder usually studies the network: they scan a machine to learn which services are running, try to overwhelm a server so that genuine users are refused, or quietly place themselves between two devices by forging address information. Each of these actions leaves a recognisable pattern in the packets crossing the network, so a program that inspects traffic closely enough can notice the attack while it is happening [1].

An Intrusion Detection System (IDS) is software that watches activity for signs of unauthorised or abnormal behaviour and reports it to an administrator [2], [3]. An IDS is a passive sensor. Unlike a firewall, which decides what traffic is allowed through, and unlike an Intrusion Prevention System, which blocks what it detects, an IDS only observes and raises an alert. This makes it safe to run on a live network, because it cannot accidentally cut off genuine traffic. A Network-based IDS (NIDS) watches the packets on a network segment, whereas a Host-based IDS examines the logs and processes of a single machine. The system described in this report is a NIDS.

Detection is usually divided into two styles. Signature-based detection matches traffic against known malicious patterns, and anomaly-based detection builds a model of normal behaviour and raises an alarm when traffic deviates from it [2]. Large production tools such as Snort [4] and Zeek [5] are powerful, but they depend on extensive rule sets written in their own languages and need expert configuration, and their output is usually a text log rather than a live picture. For a small network, and for a student who must be able to explain every decision the program makes, a simpler and more transparent approach is more useful.

The three attacks addressed in this project are among the most common first steps an intruder takes on a local network. A port scan is a rapid series of connection attempts to many ports of a host, used to discover which services are running. A SYN flood is a denial-of-service attack that sends a large number of half-open connection requests so that a server's connection table fills up and genuine users are refused. ARP spoofing is the act of forging address-resolution replies so that a victim sends its traffic to the attacker instead of to the real gateway. All three can be recognised from the headers of the packets alone, without reading any private data, which is why a lightweight, header-only detector is able to catch them.

The cost of leaving such attacks undetected is real even on a small network. A successful port scan tells an attacker exactly which services to target next. A SYN flood can make a shared

printer, a file server or a home router unreachable for everyone. An ARP-spoofing attack lets an intruder read or alter traffic that the victim believes is private, including login pages that are not properly protected. Because these attacks are cheap to launch with freely available tools, the ability to notice them quickly is valuable even in a setting as modest as a hostel room or a teaching laboratory. This project aims to put that ability within reach of an ordinary user on ordinary hardware.

1.2 Motivation

The motivation for this project is the gap between the tools that professionals use and the needs of a small network. A student living in a hostel, a family at home, or a teacher running a laboratory has no realistic way to see an attack as it happens. Professional tools are either passive analysers that raise no alarm, or heavy engines that need expert configuration. At the same time, the subject is an excellent learning vehicle: building a detector from first principles forces a clear understanding of how the Transmission Control Protocol, the Address Resolution Protocol and packet capture actually work. The aim, therefore, was to build something small enough to understand completely, useful enough to run on a real network, and clear enough that every alert it raises can be justified.

1.3 Problem Statement

Small networks are the least protected part of the Internet. A home router, a hostel wireless network or a departmental laboratory rarely has any monitoring beyond the router's own log, so a port scan or an address-spoofing attempt can continue for its whole duration without being noticed. The tools that do exist each solve only part of the problem. Packet analysers such as Wireshark [18] capture and decode traffic in great detail, but they are designed for manual, after-the-fact study and raise no alerts of their own. Production detection engines such as Snort and Zeek are effective but heavy, and they report to log files or separate consoles that a beginner cannot easily read in real time. Machine-learning prototypes report high accuracy on benchmark datasets, but their decisions are difficult to explain and they are seldom trained on the traffic of the network they are meant to protect [11]. There is, therefore, a need for a system that is light enough to run on a laptop, transparent enough that every alert can be explained line by line, and immediate enough that a threat appears on a screen within about a second of being detected.

1.4 Objectives

The objectives of this mini project are:

1. To capture live TCP, UDP and ARP packets from a local wireless or loopback interface and decode their protocol fields in real time.

2. To design and implement a transparent, rule-based detection engine for three attack classes: port scanning, SYN-flood denial of service, and ARP spoofing.
3. To build an event pipeline in which the sensor sends each alert as a JSON message over HTTP to a Node.js and Express server that validates, timestamps and logs it.
4. To develop a browser dashboard that receives live packet statistics and alerts through Socket.IO and displays them as a chart, a colour-coded banner and an alert table without the page being refreshed.
5. To verify the system with automated unit tests for every rule and with controlled, self-generated attack traffic, and to measure its detection latency and false-alarm behaviour.

1.5 Scope of the Project

The system monitors the traffic that is visible to one network interface, namely broadcast traffic, traffic addressed to the monitoring host, and traffic on the loopback interface used for self-testing. It detects the three attack classes listed above using deterministic rules whose thresholds are read from a configuration file. It does not inspect encrypted payloads, because all three rules work on header fields alone, and it does not block traffic, because it is a detection system and not a prevention system. The prototype is intended for small networks and for teaching, not as a replacement for a full production IDS. The rule engine is written so that further rules can be added as small, independent functions, and a list of planned extensions is given in Chapter 7.

1.6 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 reviews existing intrusion detection systems and the techniques used to detect the three chosen attacks, and identifies the gap this project fills. Chapter 3 presents the requirement analysis and the complete system design, including the architecture, data-flow, use-case, sequence, component and deployment diagrams, and the data design. Chapter 4 describes the implementation of each module, with the key algorithms, code excerpts and the reasoning behind them. Chapter 5 explains the testing strategy and presents the unit and integration test results. Chapter 6 shows the results obtained from the running system and discusses its performance and limitations. Chapter 7 concludes the report and lists the future scope. The References and an Appendix containing the full source code follow.

CHAPTER 2: LITERATURE REVIEW / EXISTING SYSTEM

2.1 Intrusion Detection Systems

The idea of detecting intrusions by examining system activity was formalised by Denning [3], whose model compared audit records against profiles of normal behaviour together with a set of rules. Almost every later IDS, including the one in this project, follows that same rule-and-profile structure. Scarfone and Mell, in the United States National Institute of Standards and Technology guide to intrusion detection and prevention systems [2], later classified such systems along two axes. The first axis is placement: a network-based system watches packets on a segment, while a host-based system examines one machine's own logs and processes. The second axis is method: a signature-based system matches traffic against known patterns, while an anomaly-based system learns what is normal and flags deviations. This vocabulary is used throughout the present report. The system described here is a network-based, signature-style detector, in which each signature is a simple threshold on a counter rather than a fixed byte pattern. Fig. 2.1 places the present system within this standard taxonomy.

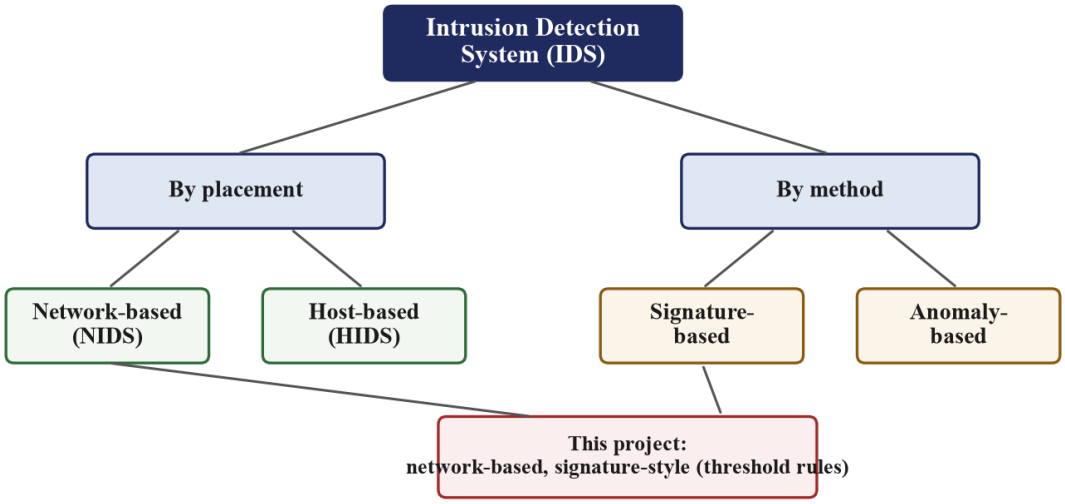


Fig. 2.1 Taxonomy of intrusion detection systems and the place of this project

A related distinction is between detection and prevention. An Intrusion Prevention System sits in the path of the traffic and can drop it, whereas an IDS observes a copy and only reports. Detection is the safer starting point for a student project, because a detector cannot accidentally break the network it watches. The present system is purely a detector; a possible prevention extension is noted in the future scope.

2.2 Existing Tools

Three widely used tools illustrate the range of existing solutions. Wireshark [18] is a packet analyser that captures traffic and decodes it layer by layer in exceptional detail. It is invaluable for manual investigation, and it is used in this project to confirm that the sensor sees the same packets that Wireshark sees, but it is not a detector: it never raises an alert on its own. It is designed for an expert who already knows what to look for and has time to study a capture after the event.

Snort, introduced by Roesch in 1999 as a lightweight network intrusion detection system [4], is a single-threaded engine that decodes packets and matches them against a text rule set. Snort proved that a rule-based sensor on commodity hardware could be effective, and it remains the reference against which small detectors are compared. However, it depends on large rule sets written in its own syntax, it must be tuned carefully to avoid false alarms, and it reports to log files or a separate console rather than to a live screen, so a beginner cannot easily see what is happening in real time.

Bro, created by Paxson [5] and now called Zeek, moved detection into a scripting language that reacts to protocol events rather than byte patterns. This makes it extremely powerful for research and for large networks, but it has a steep learning curve and, like Snort, no built-in real-time web dashboard. Both Snort and Zeek are far larger than a small network needs, and both assume an administrator who can read and write their configuration.

2.3 Attack Detection Techniques

For port scanning, Jung and co-authors [6] showed that a scanner can be identified quickly from the proportion of its connection attempts that fail, because legitimate clients rarely contact closed ports. Their sequential-hypothesis-testing method is accurate but statistically involved. The simpler rule adopted in this project, which counts the number of distinct destination ports contacted by one source within a short window, is the approach used by Snort's scan pre-processor. It is chosen here because it is easy to explain, easy to test with crafted packets, and sufficient to catch the fast scans that common tools produce by default.

For SYN floods, Wang, Zhang and Shin [7] detected the attack from the imbalance between SYN packets, which open connections, and the FIN packets that close them. RFC 4987 [8] documents the attack in detail and describes mitigations such as SYN cookies, which let a server avoid storing state for a half-open connection until the handshake completes. The rule used in this project applies the same imbalance idea in a simpler form: it counts half-open handshakes to a destination within the window and compares them with the number of completed handshakes, raising an alert when there are many of the former and almost none of the latter.

ARP spoofing exploits the stateless, unauthenticated design of the Address Resolution Protocol defined in RFC 826 [9]. Because a host accepts an address-resolution reply even when it never sent a request, an attacker can repeatedly announce that the gateway's address belongs to the attacker's machine, so that the victim sends its traffic through the attacker. Ramachandran and Nandi [10] reviewed passive and active methods of detecting this. The passive method chosen here, watching for a change in the hardware address bound to a known address, is the principle used by the well-known arpwatch utility, and needs no cooperation from other hosts.

2.4 Rule-Based Detection versus Machine Learning

Much recent research applies machine learning to intrusion detection, often reporting strong results on the KDD Cup 99 dataset analysed by Tavallaee and co-authors [12]. Sommer and Paxson [11], however, explained why such systems are seldom used in practice: false positives are costly, the decisions are hard to interpret, and the data used for training rarely matches live traffic. These observations directly motivate the design choice made in this project. A deterministic rule engine needs no training data, runs in real time on a laptop, and, most importantly for a mini project that must be defended in a viva, produces alerts that can each be traced to a specific counter and threshold. Transparency, rather than raw accuracy on a benchmark, is the property that matters for a small-network sensor, and it is the property this project prioritises.

2.5 Comparison and Identified Gap

Table 2.1 compares the existing tools with the system proposed in this report. The gap is clear. None of the existing options is, at the same time, light enough to run on a student laptop, transparent enough that every alert can be explained, and immediate enough to show a threat on a live dashboard. The proposed system occupies exactly that gap. Its contribution is not a new detection algorithm but a careful combination: deterministic, explainable rules, a language-neutral alert pipeline, and a real-time web dashboard, in a package small enough to be understood in full.

Table 2.1 Comparison of existing tools with the proposed system

Tool	Detects attacks	Real-time alert	Live dashboard	Suited to small networks
Wireshark [18]	No (manual study)	No	No	Partly
Snort [4]	Yes	Logs	No	Heavy to configure
Zeek [5]	Yes	Logs / scripts	No	Steep learning curve

Tool	Detects attacks	Real-time alert	Live dashboard	Suited to small networks
Machine-learning IDS [11]	Yes	Varies	Rarely	Needs training data
Proposed system	Yes (three rules)	Yes (< 1 s)	Yes (built in)	Yes

2.6 Summary

The literature shows a mature field with powerful but heavy tools at one end and promising but opaque machine-learning methods at the other. For the specific setting of a small network and a student who must explain the system, a transparent, rule-based, real-time detector with a built-in dashboard is the most appropriate choice. The design of such a system is presented in the next chapter.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

The requirements were gathered from the problem statement and from the behaviour of the three chosen attacks. They are divided into functional requirements, which describe what the system must do, and non-functional requirements, which describe the qualities it must have. The functional requirements are listed in Table 3.1 and the non-functional requirements in Table 3.2. Together they define exactly what the finished system is expected to achieve, and they are revisited in Chapter 6 when the results are assessed.

Among these requirements, a small number are the most important and were treated as non-negotiable. The three detection requirements, FR3 to FR5, define the core purpose of the system, and the transparency and latency requirements, NFR1 and NFR2, define the qualities that distinguish it from the existing tools reviewed in Chapter 2. The remaining requirements add convenience and robustness. Keeping this priority in mind guided several design decisions, such as the choice of simple, explainable rules over more accurate but opaque methods, and the decision to send alerts over a separate thread so that detection is never delayed.

Table 3.1 Functional requirements

ID	Requirement
FR1	Capture live TCP, UDP and ARP packets from the selected interface and the loopback adapter.
FR2	Count packets of each protocol and report the counts once per second.
FR3	Detect a port scan from the number of distinct destination ports contacted by one source in the window.
FR4	Detect a SYN flood from the number of half-open handshakes to a destination in the window.
FR5	Detect ARP spoofing when a trusted address appears with a different hardware address.
FR6	Send each alert as a JSON message to the server, which validates and logs it.
FR7	Display live counts, a packet-rate chart, a colour-coded banner and an alert table in the browser.
FR8	Preserve the alert history so that it survives a page reload.

Table 3.2 Non-functional requirements

ID	Requirement
NFR1	Detection latency of about one second or less from attack to on-screen alert.
NFR2	Transparency: every alert traceable to a counter and a threshold.

ID	Requirement
NFR3	Portability: runs on an ordinary laptop with free, open-source software.
NFR4	Configurability: thresholds and the interface name read from a configuration file.
NFR5	Robustness: the sensor keeps running if the server is temporarily unavailable.

3.2 Feasibility Study

Technical feasibility was established by confirming that every component can be built from mature, free software: Python with the Scapy library and the Npcap driver for capture, and Node.js with Express and Socket.IO for the server and dashboard. A short proof-of-concept that captured and counted packets was written first to confirm that live capture works on the development laptop. Economic feasibility is clear, because all of this software is free and open source and the system runs on hardware the student already owns; the project has no monetary cost. Operational feasibility is also clear, because the system runs from three simple commands and presents its results in an ordinary web browser, so no special skill is needed to operate it once it is installed.

3.3 System Architecture

The system has four cooperating components, shown in Fig. 3.1. The Sniffer Engine captures frames from the interface and decodes their headers. The Threat Detection Module keeps per-source and per-destination counters and a table of trusted address mappings, and applies the three rules to every packet. The Event Pipeline is a Node.js server that receives alerts over HTTP, records them and forwards them. The Real-Time Dashboard is a browser page connected to the server through Socket.IO. Packets flow from left to right and alerts flow from the detector to the browser. The two halves of the system are written in different languages on purpose: Python is the most convenient language for packet capture, while Node.js with Socket.IO is the simplest way to push live updates to a browser. They communicate only through small JSON messages, so each half can be developed and tested on its own.

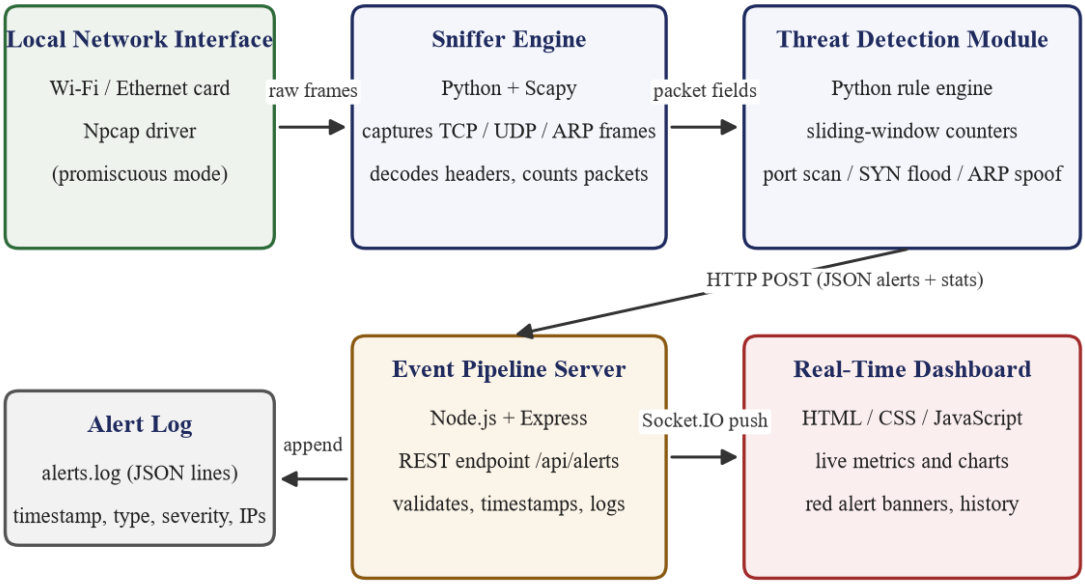


Fig. 3.1 Overall architecture of the real-time NIDS

3.4 Data Flow Diagrams

The context, or level-0, data flow diagram in Fig. 3.2 shows the whole system as a single process that sits between the network hosts and the administrator. Packets enter from the network, and live counts and alerts leave towards the administrator. This highest-level view hides all internal detail and simply states what the system consumes and produces.

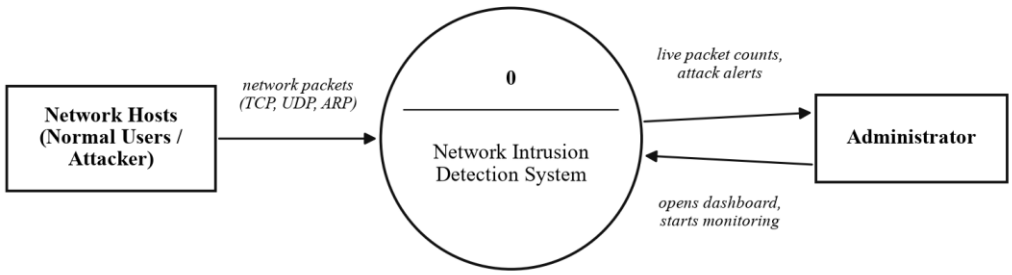


Fig. 3.2 Level-0 data flow diagram (context diagram)

The level-1 diagram in Fig. 3.3 opens the single process into five numbered processes. Process 1.0 captures packets. Process 2.0 counts them by protocol. Process 3.0 applies the detection rules and reads and writes the SYN-history data store. Process 4.0, the server, receives counts and alerts and writes each alert to the log. Process 5.0 is the dashboard, which presents the data

to the administrator. The two data stores are the SYN history, held in memory, and the alert log file on disk.

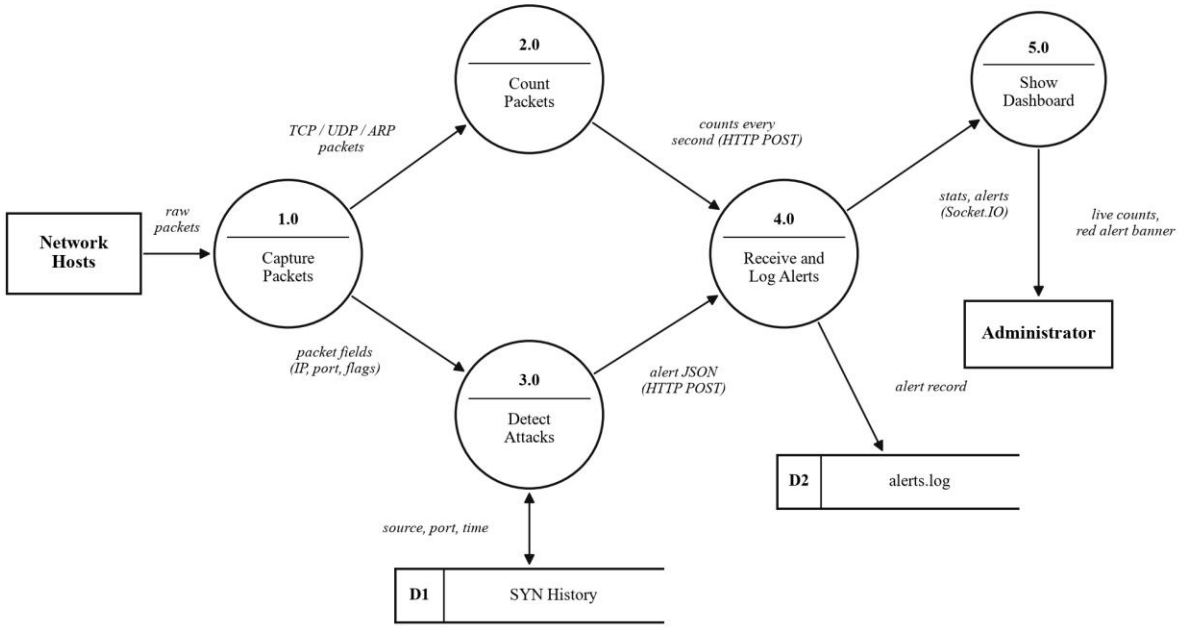


Fig. 3.3 Level-1 data flow diagram

3.5 Use-Case Diagram

The use-case diagram in Fig. 3.4 identifies two actors. The administrator starts monitoring, views the live packet counts and the real-time alerts, checks the alert log, and may change the detection thresholds. The attacker is an external actor whose traffic triggers the detection use case, which in turn notifies the administrator through the real-time alert display. The diagram makes explicit that the attacker never interacts with the dashboard directly; the attacker's only effect is to generate traffic that the system observes.

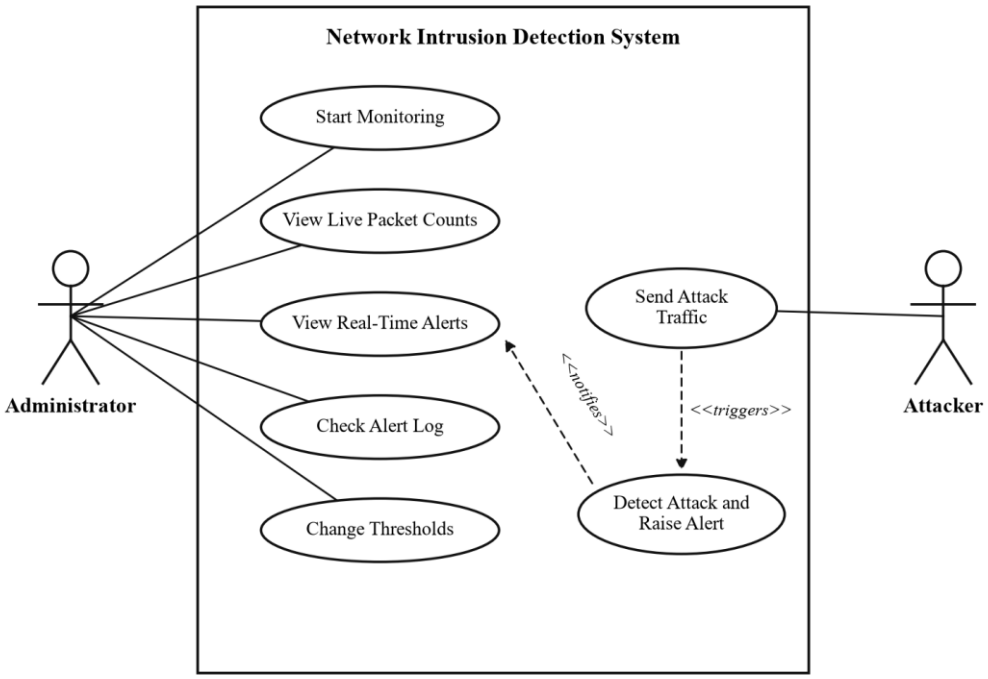


Fig. 3.4 Use-case diagram of the system

3.6 Sequence Diagram

The sequence diagram in Fig. 3.5 shows the order of events when a packet arrives. The sniffer passes the decoded fields to the detection rules, which update their windows and test the thresholds. When a rule fires, the sensor sends an HTTP POST to the server, which appends one line to the log file and emits a Socket.IO event, and the dashboard shows the banner and the new table row. In parallel, once every second, the sensor posts the packet counts, which the server forwards to the live chart. The diagram shows that the path from a packet to an on-screen alert passes through exactly four handovers, which is why the end-to-end latency is so small.

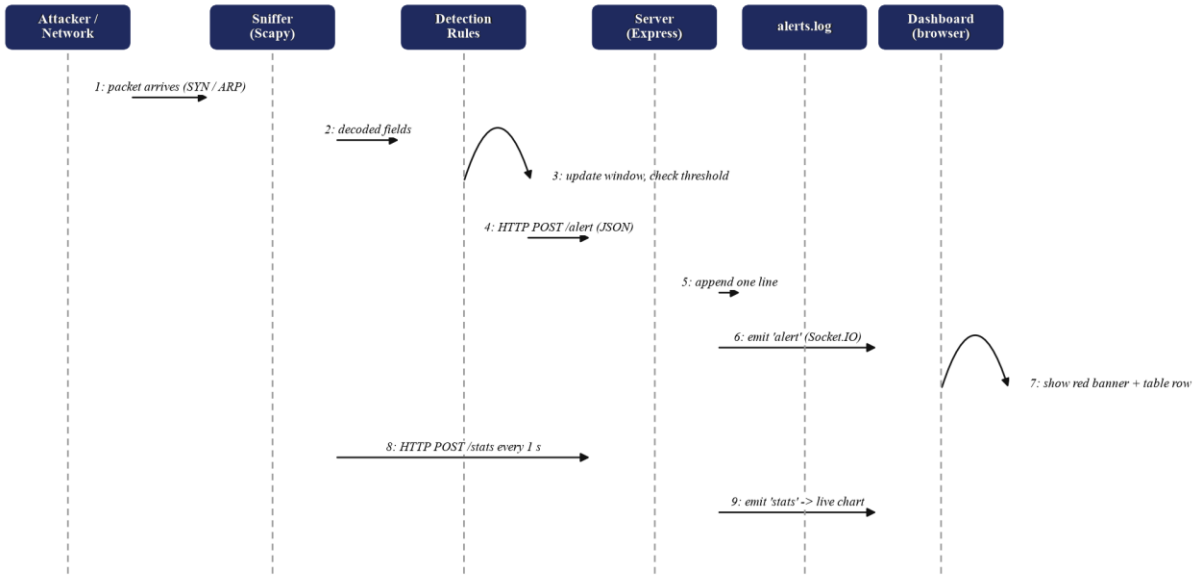


Fig. 3.5 Sequence diagram from packet arrival to on-screen alert

3.7 Component Diagram

The component diagram in Fig. 3.6 groups the code into three subsystems. The Python sensor contains the configuration loader, the capture loop, the packet parser, the rule engine, a statistics thread and a sender thread. The Node.js server contains the Express routes, the log writer and the Socket.IO hub. The browser dashboard contains the Socket.IO client, the Chart.js graph and the alert table. The subsystems communicate only through HTTP and WebSocket messages, so each can be tested on its own; for example, the server can be tested with a command-line tool that posts a sample alert, without the sensor running at all.

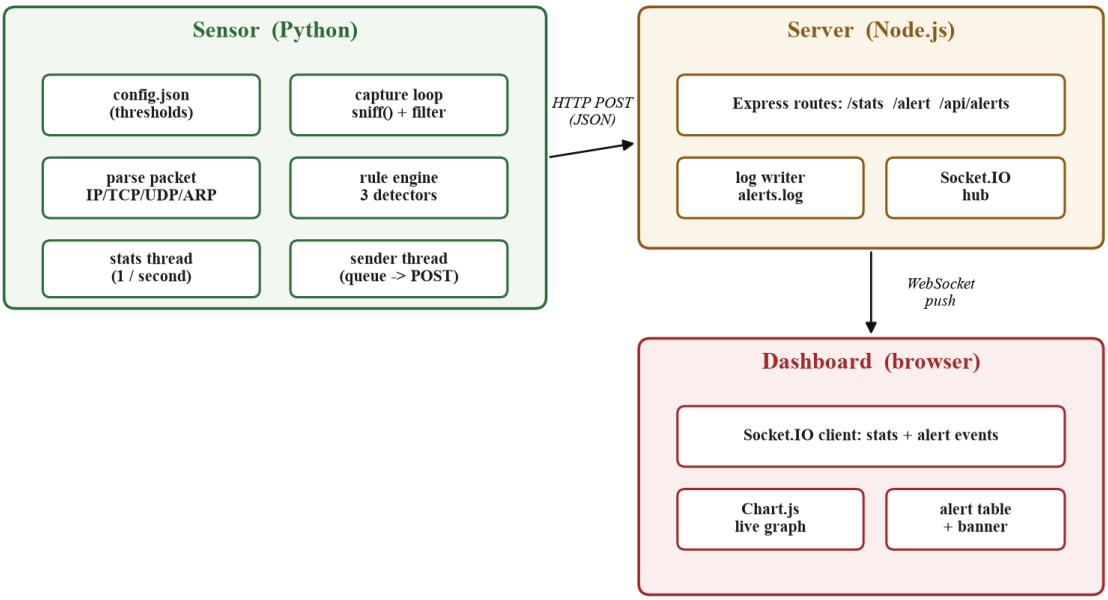


Fig. 3.6 Component diagram of the three subsystems

3.8 Deployment Diagram

The deployment diagram in Fig. 3.7 shows that all three processes run on a single laptop during development and demonstration. The Python sensor captures from the wireless adapter and the loopback adapter through the Npcap driver; the Node.js server listens on port 3000; and the dashboard runs in a browser tab on the same machine. Other devices on the same wireless network, such as a phone or a second laptop, can generate the traffic that the sensor observes. In a larger deployment the sensor and the server could run on different machines, because they communicate only over HTTP.

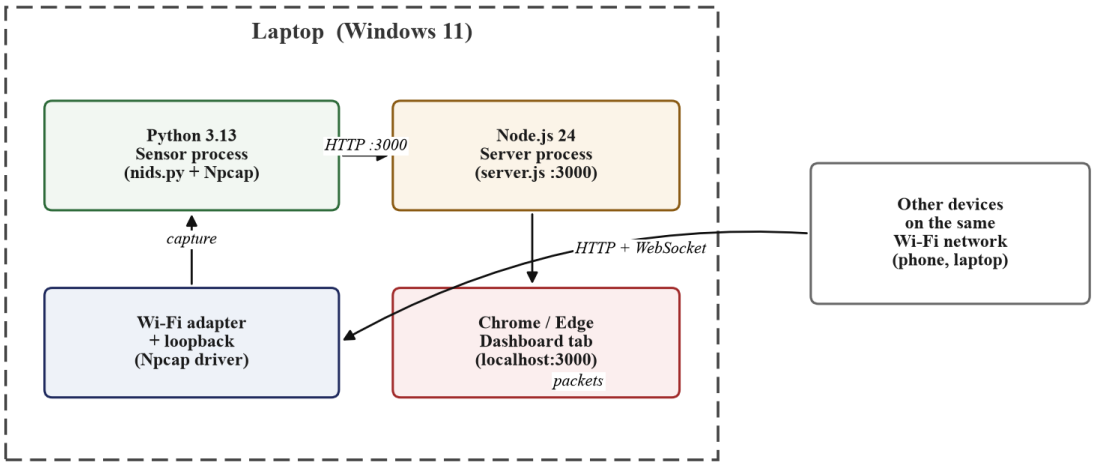


Fig. 3.7 Deployment diagram of the prototype

3.9 Data Design

The system does not use a relational database. The data it handles is small and append-only, so two simple stores are sufficient. The first is the SYN history, held in memory as a dictionary that maps each address to a double-ended queue of recent events; entries older than the window are discarded continuously, so the memory used stays bounded. The second is the alert log, a plain text file in which each alert is written as one line of JSON, so that it is both human-readable and easy to load later. The structure of an alert record is given in Table 3.3, and the packet fields the detector reads are listed in Table 3.4. The relationship of these fields to the layers of a packet is shown in Fig. 3.8. Storing alerts in a lightweight database such as SQLite is listed as future work; for the present scope a log file is simpler and fully sufficient.

Table 3.3 Fields of an alert record

Field	Meaning
type	Attack class: Port Scan, SYN Flood or ARP Spoofing.
severity	medium, high or critical, shown as a colour on the dashboard.
src	Source address responsible for the traffic.
dst	Destination (victim) address.
detail	The counter value and threshold that fired the rule.
time	Clock time at which the alert was raised.
latency_ms	Time from packet capture to alert, used for evaluation.
date	Calendar date, added by the server when the alert is logged.

Table 3.4 Packet fields read by the detector

Field	Source layer	Used by
Source / destination IP	IP header	all rules
Source / destination port	TCP header	port scan, SYN flood
TCP flags (SYN, ACK, RST)	TCP header	port scan, SYN flood
ARP operation code	ARP header	ARP spoofing
Sender IP / hardware address	ARP header	ARP spoofing
Capture timestamp	added by Npcap	sliding windows, latency

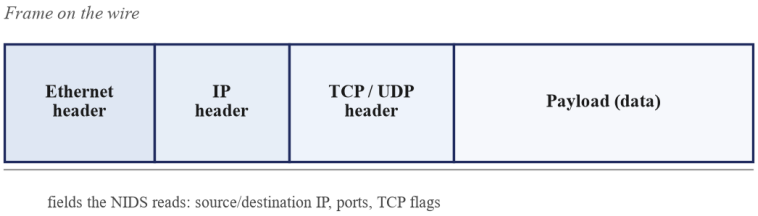


Fig. 3.8 Layers of a captured frame and the fields the NIDS reads

3.10 Design Summary

The design can be summarised as a short pipeline with two memory structures. Packets enter the sniffer, are parsed into a small set of fields, and are passed to three independent rules. Each rule keeps only the events of the last few seconds in a double-ended queue and compares a simple count against a threshold. When a threshold is crossed, a compact alert record travels, as JSON, to a server that logs it and forwards it to a browser. Table 3.5 is a short data dictionary that names every important variable in the detector and gives its purpose; it serves as a quick reference for the implementation described in the next chapter.

Table 3.5 Data dictionary of the detection module

Name	Structure	Purpose
counts	dictionary of integers	packets seen of each protocol in the current second
syn_by_src	address to queue of (time, port)	recent SYNs per source, for the port-scan rule
syn_by_dst	address to queue of (time, port)	recent SYNs per destination, for the SYN-flood rule
ack_by_dst	address to queue of (time, port)	recent completed handshakes per destination

Name	Structure	Purpose
trusted	address to hardware address	known-good gateway mapping, for the ARP rule
last_alert	(type, subject) to time	time of the last alert, for the cool-down
outbox	thread-safe queue	alerts and counts waiting to be sent to the server

CHAPTER 4: IMPLEMENTATION

4.1 Development Environment

The system was developed and tested on the environment listed in Table 4.1. All of the software is free or open source, apart from the operating system and the office tools used to prepare this report. The sensor is about one hundred and twenty lines of Python, the server about thirty lines of JavaScript, and the dashboard about one hundred lines of HTML, CSS and JavaScript, so the whole system is small enough to be read and understood in a single sitting.

The code is organised into a small, predictable folder layout. The sensor folder holds the Python configuration file, the main program and the two short scripts used to generate test traffic. The server folder holds the Node.js program and a public sub-folder with the dashboard page. A separate tests folder holds the automated unit tests, and a design folder holds the diagrams and screenshots used in this report. This separation keeps the two halves of the system, the Python sensor and the JavaScript server, cleanly apart, and it mirrors the structure of the appendix in which the source code is listed.

Table 4.1 Development environment

Component	Detail
Processor	Intel Core i7-11800H, 8 cores, 2.30 GHz
Memory	8 GB RAM
Operating system	Windows 11 Home, 64-bit
Capture	Scapy 2.8 over Npcap 1.89
Sensor language	Python 3.13
Server	Node.js 24 with Express 5 and Socket.IO 4
Dashboard	HTML5, CSS3, JavaScript, Chart.js 4
Editor and version control	Visual Studio Code, Git

4.2 Configuration Module

All values that might change are kept in a small file, config.json, rather than written into the code. These include the interface name, the server address, the gateway address and its hardware address, the window length, the port-scan and SYN-flood thresholds, and the cool-down period. The sensor reads this file once at start-up, as shown below. Keeping the thresholds in one place satisfies requirement NFR4 and makes it easy to tune the system during testing or to demonstrate a change during the viva. The configuration parameters are listed in Table 4.2.

Loading the configuration at start-up (sensor/nids.py)

```
cfg = json.load(open(os.path.join(HERE, "config.json")))
WINDOW = cfg["window_seconds"]
COOLDOWN = cfg["cooldown_seconds"]
```

Table 4.2 Configuration parameters

Parameter	Default	Purpose
window_seconds	5	Length of the sliding time window.
port_scan_ports	15	Distinct-port threshold for a port scan.
syn_flood_syns	100	Half-open-handshake threshold for a SYN flood.
cooldown_seconds	30	Quiet period after an alert for the same subject.
interface	Wi-Fi	Network adapter to capture from.
gateway	192.168.29.1	Gateway address whose hardware address is trusted.

4.3 Packet Capture and Parsing

Capture is performed by Scapy's sniff function [13], which is given the interface list, a callback, and a Berkeley Packet Filter, as shown below. The filter runs inside the Npcap driver [14] so that Python receives only the relevant packets and never sees the sensor's own traffic to the dashboard on port 3000. The store option is set to false so that captured packets are not kept in memory, which keeps the memory use constant no matter how long the sensor runs.

Starting live capture (sensor/nids.py)

```
sniff(iface=[cfg["interface"], conf.loopback_name],
      filter="(tcp or udp or arp) and not port 3000",
      prn=handle, store=False)
```

For every captured packet the callback, named handle, increments the protocol counter and then, depending on the layers present, calls the TCP rule or the ARP rule. Scapy exposes each layer as an object, so a field such as the destination port is read simply as the dport attribute of the TCP layer, and the TCP flags are compared directly against the string for a lone SYN. The counters are reset to zero once a second by a separate thread, described in Section 4.5.

The per-packet callback (sensor/nids.py)

```
def handle(packet):
    for name, layer in (("TCP", TCP), ("UDP", UDP), ("ARP", ARP)):
        if packet.haslayer(layer):
            counts[name] += 1
    if packet.haslayer(IP) and packet.haslayer(TCP):
        check_tcp(packet)
    elif packet.haslayer(ARP):
        check_arp(packet)
```

4.4 Detection Rules

The three rules share a common structure. Each keeps a small amount of state in dictionaries of double-ended queues, trims that state to the sliding window on every packet, tests a threshold, and calls a single alert function when the threshold is crossed. The sliding window is implemented by the small helper shown below, which removes events older than the window from the front of a queue. The alert function enforces the cool-down, builds the alert record and places it on a queue for sending. The thresholds are summarised in Table 4.3.

The sliding-window helper and the alert function (sensor/nids.py)

```
def trim(history, now):
    while history and now - history[0][0] > WINDOW:
        history.popleft()

def alert(kind, severity, src, dst, detail, packet, subject):
    now = float(packet.time)
    if now - last_alert.get((kind, subject), 0) < COOLDOWN:
        return
    last_alert[(kind, subject)] = now
    record = {"type": kind, "severity": severity, "src": src,
              "dst": dst, "detail": detail,
              "time": time.strftime("%H:%M:%S", time.localtime(now))}
    outbox.put(("/alert", record))
```

4.4.1 Port-Scan Rule

For each source address the rule keeps a queue of (time, destination port) pairs, fed only by packets whose only TCP flag is SYN. On each such packet it appends the new pair, trims the queue to the window, and counts the distinct ports that remain by placing them in a set. If the count exceeds the port-scan threshold a medium-severity alert is raised. Only lone SYN packets are counted because a scanner sends one SYN to each port it probes, whereas ordinary browsing opens connections to only a few ports on any one host and so never reaches the threshold. The code is shown below and the complete logic as a flowchart in Fig. 4.1.

Port-scan detection (sensor/nids.py, inside check_tcp)

```
if tcp.flags == "S":
    history = syn_by_src[ip.src]
    history.append((now, tcp.dport))
    trim(history, now)
    ports = {port for _, port in history}
    if len(ports) > cfg["port_scan_ports"]:
        alert("Port Scan", "medium", ip.src, ip.dst,
              "%d ports in %d s" % (len(ports), WINDOW), packet, ip.src)
```

As a worked example, suppose a scanner sends SYN packets to ports 20, 21, 22 and so on up to port 40, all within one second. After the sixteenth distinct port the set contains sixteen ports, which is greater than the threshold of fifteen, so the rule raises one alert that reports sixteen ports in five seconds. A cool-down then suppresses further alerts from the same source for thirty seconds, so a scan of a thousand ports produces a single alert rather than hundreds.

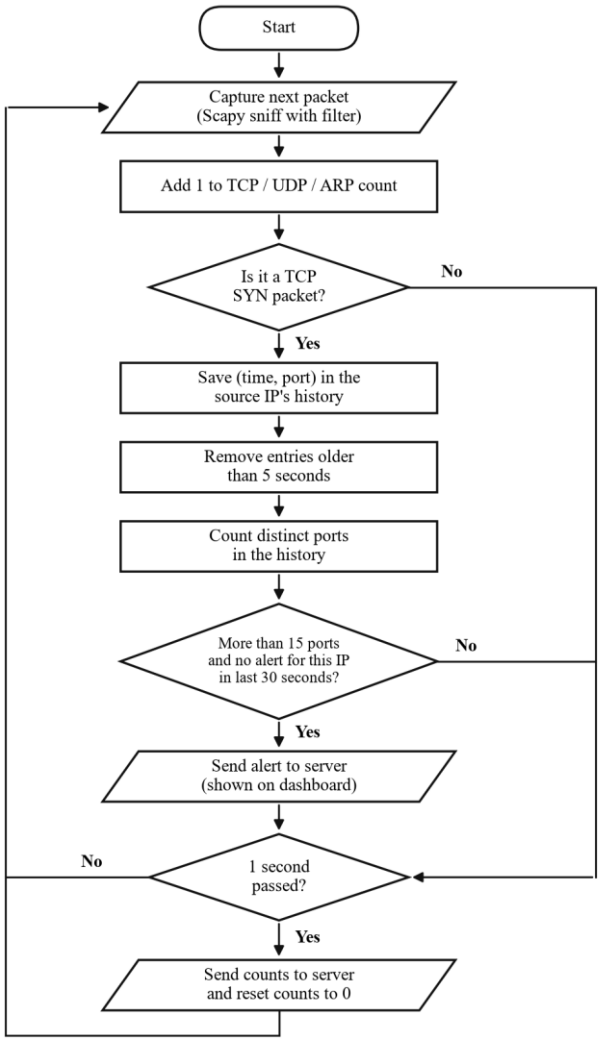


Fig. 4.1 Flowchart of packet processing and port-scan detection

4.4.2 SYN-Flood Rule

For each destination address the rule keeps a queue of the times of incoming SYN packets and a separate queue of completed handshakes, where a completed handshake is recognised from a packet that has the ACK flag set and the SYN flag clear. On each SYN it trims both queues to the window and compares the number of SYNs with the number of completions, as shown below. If the number of SYNs is at least the flood threshold, the traffic is aimed at very few ports, and the completions are fewer than one tenth of the SYNs, a high-severity alert is raised. This follows the imbalance principle of Wang and co-authors [7] and RFC 4987 [8]: a genuine burst of connections completes its handshakes, whereas a flood does not, because the sources are forged or the attacker ignores the server's reply.

SYN-flood detection (sensor/nids.py, inside check_tcp)

```
syns = syn_by_dst[ip.dst]
syns.append((now, tcp.dport))
trim(syns, now)
```

```
trim(ack_by_dst[ip.dst], now)
completed = len(ack_by_dst[ip.dst])
target_ports = {port for _, port in syns}
if len(syns) >= cfg["syn_flood_syns"] and len(target_ports) <= 3 \
        and completed < len(syns) / 10:
    alert("SYN Flood", "high", ip.src, ip.dst,
          "%d SYN, %d completed" % (len(syns), completed), packet, ip.dst)
```

For a flood the cool-down is keyed on the victim rather than the source, because a real flood arrives from many forged source addresses at once; keying on the source would produce one alert per forged address, which is exactly the flood of alerts the cool-down is meant to prevent. This subtlety was discovered during unit testing, when a flood from many sources produced many alerts, and the fix is captured by the subject argument of the alert function.

4.4.3 ARP-Spoofing Rule

At start-up the sensor learns the gateway's hardware address, either from the configuration file or by a normal address lookup, and stores it in a table of trusted mappings. For every ARP reply the rule checks whether the sender's address is in the trusted table with a different hardware address, as shown below. If it is, a critical alert is raised, because an attacker is claiming the gateway's address in order to place themselves in the middle of the victim's traffic. This passive method exploits nothing more than the fact that the Address Resolution Protocol [9] accepts replies that were never requested.

ARP-spoofing detection (sensor/nids.py)

```
def check_arp(packet):
    arp = packet[ARP]
    known = trusted.get(arp.psrc)
    if arp.op == 2 and known and arp.hwsrc.lower() != known.lower():
        alert("ARP Spoofing", "critical", arp.psrc, arp.pdst,
              "MAC changed from %s to %s" % (known, arp.hwsrc),
              packet, arp.psrc)
```

Table 4.3 Detection thresholds

Rule	Condition for an alert	Severity
Port scan	More than 15 distinct destination ports from one source in 5 s	medium
SYN flood	At least 100 half-open handshakes to one destination in 5 s with few completions	high
ARP spoofing	A trusted address seen with a different hardware address	critical

4.5 Concurrency Model

Capture must never be delayed by slow work such as a network request, or packets would be missed. The sensor therefore uses three threads. The main thread runs the capture loop and the detection rules. A statistics thread wakes once every second, reads and resets the protocol counters and places them on a queue. A sender thread takes items from that queue and performs

the actual HTTP POST requests to the server. Because the detection rules only place short records on an in-memory queue, they return almost instantly, and the slower network work happens on a separate thread. If the server is unavailable the sender catches the error and continues, which satisfies requirement NFR5. The arrangement of the three threads and the queue that connects them is shown in Fig. 4.2, and the sender is shown below.

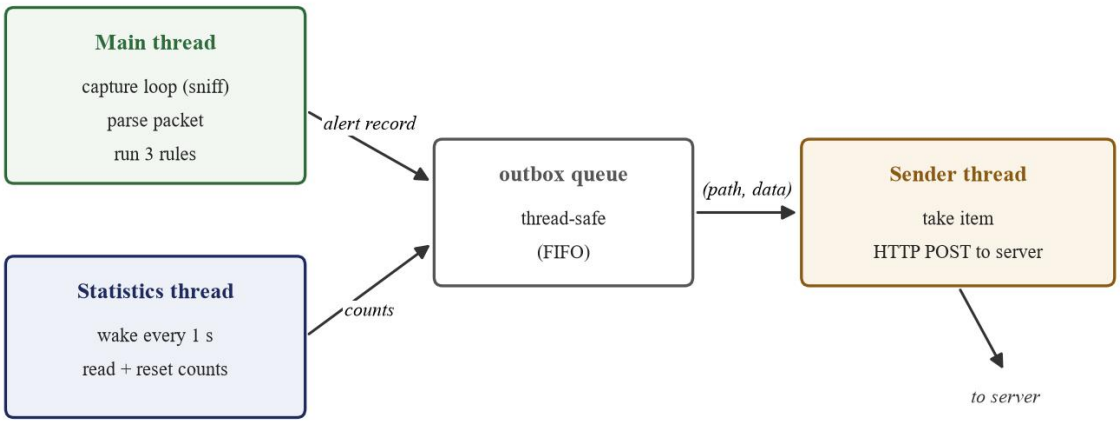


Fig. 4.2 The three threads of the sensor and the queue that connects them

The sender thread (sensor/nids.py)

```
def sender():
    while True:
        path, data = outbox.get()
        send(path, data)
        outbox.task_done()
```

4.6 Event-Pipeline Server

The server is written with the Express framework [16] on Node.js [15]. It exposes three routes, summarised in Table 4.4. A POST to /stats forwards the per-second counts to every connected browser through Socket.IO. A POST to /alert validates the alert, rejecting any message that lacks a type, a source or a severity with the status code 400, then adds the date, appends the record as one line to the log file, and emits it to the browsers. A GET to /api/alerts returns the most recent hundred alerts from the log, which the dashboard requests when it first loads so that the alert history survives a page reload. The alert route is shown below.

Table 4.4 HTTP endpoints exposed by the server

Method and path	Sent by	Purpose
POST /stats	sensor, once per second	forward packet counts to the dashboard chart
POST /alert	sensor, on each alert	validate, log and forward an alert
GET /api/alerts	dashboard, on load	return the recent alert history

Method and path	Sent by	Purpose
GET /	browser	serve the dashboard page and its assets

The alert route (server/server.js)

```
app.post("/alert", (req, res) => {
  const alert = req.body;
  if (!alert.type || !alert.src || !alert.severity) return res.sendStatus(400);
  alert.date = new Date().toISOString().slice(0, 10);
  fs.appendFileSync(LOG, JSON.stringify(alert) + "\n");
  io.emit("alert", alert);
  res.sendStatus(200);
});
```

The history route is short but important. It reads the log file, keeps the most recent hundred lines, and returns them as a JSON array, which the dashboard requests when it loads so that reopening the page does not lose the alerts already recorded:

The alert-history route (server/server.js)

```
app.get("/api/alerts", (req, res) => {
  const lines = fs.existsSync(LOG)
    ? fs.readFileSync(LOG, "utf8").trim().split("\n").filter(Boolean) : [];
  res.json(lines.slice(-100).map(line => JSON.parse(line)));
});
```

Socket.IO [17] maintains a persistent connection to each browser, which is what allows the server to push alerts the instant they occur, rather than waiting for the browser to ask. This is the key difference between the dashboard and an ordinary web page that would have to be refreshed to show new data. Keeping the server this small, around thirty lines in total, was a deliberate goal: the whole alert pipeline can be read and understood in a few minutes, which is exactly the transparency the project set out to achieve.

4.7 Real-Time Dashboard

The dashboard is a single HTML page with a small amount of CSS and JavaScript. On loading, it opens a Socket.IO connection and requests the alert history. When a statistics message arrives it updates the three protocol counters and adds a point to a Chart.js line graph of the total packet rate. When an alert message arrives it adds a row to the alert table, colours the severity as a pill, updates the per-type totals, and shows a banner coloured according to the severity for a few seconds, as shown below.

Handling an incoming alert in the browser (server/public/index.html)

```
socket.on("alert", alert => {
  addRow(alert, true);
  const banner = document.getElementById("banner");
  banner.textContent = "ALERT: " + alert.time + " " + alert.type +
      " from " + alert.src + " to " + alert.dst + " (" + alert.detail + ")";
  banner.className = alert.severity;
  banner.hidden = false;
  setTimeout(() => banner.hidden = true, 6000);
});
```

All text taken from the network is inserted with the textContent property rather than as HTML, so that data from the wire can never inject markup or script into the page. This is a deliberate security choice: an attacker who controlled, say, a source address string could otherwise try to smuggle a script into the dashboard, and using textContent closes that possibility.

When the page first loads it fetches the recent alert history from the server and adds each row to the table, so that reopening or refreshing the dashboard does not lose the alerts already recorded. The statistics handler updates the counters and feeds the live chart, keeping only the most recent sixty points so that the graph scrolls smoothly:

Updating the live chart from a statistics message (server/public/index.html)

```
socket.on("stats", data => {
  for (const name in data) document.getElementById(name).textContent = data[name];
  chart.data.labels.push("");
  chart.data.datasets[0].data.push(data.TCP + data.UDP + data.ARP);
  if (chart.data.labels.length > 60) {
    chart.data.labels.shift(); chart.data.datasets[0].data.shift();
  }
  chart.update();
});
```

4.8 Starting and Running the System

The system is started from three terminals. The first starts the server, the second starts the sensor, and the third is used to generate test traffic. The commands are short and need no configuration beyond the one-time installation of Npcap, Python with Scapy, and the Node packages:

Starting the server, the sensor and a test

```
# terminal 1: start the dashboard server
cd server
node server.js

# terminal 2: start the sensor (captures live traffic)
cd sensor
python nids.py

# terminal 3: generate a test port scan
cd sensor
python scan_test.py
```

After the server starts it prints the dashboard address, http://localhost:3000, which is opened in a browser. Once the sensor is running the protocol counters begin to move and the chart fills in. Running the test script produces a port-scan alert within about a second: a coloured banner appears, a new row is added to the table, and the same record is written to the alert log. Stopping the sensor or the server is done with the usual interrupt key, and the sensor shuts its threads down cleanly.

4.9 Statistics Reporting and Start-up

Two further pieces of the sensor complete the picture. The statistics thread, shown below, wakes once a second, copies and clears the protocol counters, prints them for the operator, and places them on the outbox queue so that the sender thread forwards them to the dashboard. Copying and clearing the counters in one step means that no packet is counted twice or missed between reports.

The statistics thread (sensor/nids.py)

```
def stats():
    while True:
        time.sleep(1)
        snapshot = dict(counts)
        for name in counts:
            counts[name] = 0
        print(snapshot)
        outbox.put(("/stats", snapshot))
```

The send function performs the actual HTTP request and is deliberately forgiving: if the server cannot be reached it prints a short message and returns, rather than raising an error that would stop capture. This is what allows the sensor to keep running when the server is not available, satisfying requirement NFR5.

Sending data to the server, tolerant of failure (sensor/nids.py)

```
def send(path, data):
    request = urllib.request.Request(cfg["server"] + path,
        json.dumps(data).encode(), {"Content-Type": "application/json"})
    try:
        urllib.request.urlopen(request, timeout=2)
    except OSError:
        print("server not reachable")
```

At start-up the sensor learns the gateway's hardware address, which the ARP rule treats as trusted. If the address cannot be learned, the ARP rule is simply disabled rather than guessing, so the system never raises a false ARP alert on start-up:

Learning the trusted gateway at start-up (sensor/nids.py)

```
def learn_gateway():
    mac = cfg["gateway_mac"] or getmacbyip(cfg["gateway"])
    if mac:
        trusted[cfg["gateway"]] = mac
        print("trusted gateway", cfg["gateway"], "at", mac)
    else:
        print("gateway MAC unknown, ARP rule is off")
```

The sensor itself is written in the standard Python library wherever possible [19], using only Scapy as an external dependency for capture; the configuration loader, the queues, the threads and the HTTP client are all standard-library features. This keeps the number of moving parts small and the installation simple.

4.10 Security Considerations

Although the system is itself a security tool, it must not become a weakness in the network it watches, so several precautions were taken in its implementation. The communication between the sensor and the server uses ordinary HTTP [20], [23] carrying small JSON [24] messages; because both run on the same machine during the demonstration, the traffic never leaves the host. The dashboard inserts every value that comes from the network using the textContent property rather than as HTML, which prevents a crafted address or detail string from injecting a script into the page, the class of flaw known as cross-site scripting [22]. The live chart is drawn with the Chart.js library [21], which receives only numbers, not markup.

On the server side, the alert route validates every incoming message and rejects anything that lacks the required fields with the status code 400, so malformed or partial data is never written to the log. The Berkeley Packet Filter applied at capture time also acts as a first line of defence, because it discards everything except the TCP, UDP and ARP traffic the rules actually need, which both improves performance and reduces the amount of data the sensor has to reason about.

One limitation remains, and it is stated plainly. The server currently accepts an alert from any client on the same machine, because it does not authenticate the sensor. On a single laptop this is acceptable, but a deployment in which the sensor and server run on different machines should require the sensor to prove its identity, for example with a shared secret key sent in a header. This improvement is listed in the future scope in Chapter 7.

CHAPTER 5: TESTING

5.1 Testing Strategy

The system was tested at two levels. Unit testing checks each detection rule in isolation by feeding hand-crafted packets directly to the rule and asserting on the alerts produced; this proves the logic without depending on the network. Integration testing then runs the whole system, generates real attack traffic against the host, and confirms that an alert travels all the way from the sensor to the browser and into the log file. Both positive cases, in which an alert must appear, and negative cases, in which no alert must appear, were tested, because a detector that raises false alarms on ordinary traffic is of little use.

The testing philosophy was that every rule must be proved to fire when it should and, equally, to stay silent when it should not. A detector that catches every attack but also fires on ordinary web browsing is worse than useless, because its alerts would soon be ignored. For this reason the ten unit tests are deliberately balanced between positive cases, which supply attack traffic and expect an alert, and negative cases, which supply ordinary or borderline traffic and expect silence. The negative cases also guard against the subtle errors that are easy to introduce, such as counting reply packets as part of a scan or mistaking a port scan for a flood.

5.2 Unit Testing

Ten unit tests were written with Python's standard unittest framework. Each test builds packets with Scapy's own constructors, gives them explicit timestamps, and passes them to the packet handler, then inspects the queue of alerts. Because the timestamps are supplied by the test rather than by the clock, the sliding-window and cool-down behaviour can be checked exactly and repeatably. The tests and their results are listed in Table 5.1. All ten passed. The negative tests are as important as the positive ones: they confirm that reply packets, slow activity below the threshold, completed handshakes and genuine address replies do not raise alarms, and that a port scan is not mistaken for a flood. The tests can be re-run at any time with a single command, which makes it safe to change the code and confirm that nothing has broken.

Table 5.1 Unit test cases and results

Test	What it checks	Result
Port scan fires	20 SYNs to 20 ports raise exactly one medium alert	Pass
Few ports	10 ports raise no alert	Pass
SYN-ACK ignored	Reply packets are not counted as a scan	Pass
Slow scan	One port per second stays under the window	Pass

Test	What it checks	Result
Cool-down	A repeat within 30 s raises no second alert	Pass
SYN flood fires	120 half-open SYNs raise one high alert	Pass
Completed handshakes	Full handshakes are not a flood	Pass
Scan not a flood	Many ports report only a port scan	Pass
ARP spoof fires	A changed gateway MAC raises a critical alert	Pass
Genuine ARP	Normal ARP traffic raises no alert	Pass

5.3 Integration Testing

For integration testing the server and sensor were started together and the dashboard was opened in a browser. A port scan was generated with a short script that opens connections to a hundred ports of the host, and a SYN flood with a script that makes several hundred rapid connection attempts to a single port; both target only the local host, so no other machine is affected. The ARP rule cannot be exercised on a single machine, because the loopback adapter does not carry address-resolution frames, so it is covered by the unit test and would be demonstrated on a live network with a second device. The integration results are given in Table 5.2. In every case the alert reached the dashboard and the log within a few milliseconds of the threshold being crossed.

Table 5.2 Integration test results

Scenario	Expected	Observed
Port scan from host	One medium alert on the dashboard and in the log	As expected, 16 ports reported
SYN flood to host	One high alert on the dashboard and in the log	As expected, 100+ SYNs reported
Normal browsing	No alert during ordinary web traffic	No alert over the observation period
Repeat scan in 30 s	No duplicate alert during the cool-down	No duplicate raised
Server stopped	Sensor keeps running, reports it cannot reach the server	Sensor continued without crashing

5.4 Sample Input and Output

As an example, when the port-scan script was run against the host, the sensor printed an alert record of the form shown below, and the same record appeared as one line in the alert log and as a new row on the dashboard. The detail field records the exact counter value, sixteen ports, that crossed the threshold, so the alert explains itself:

Sample alert record (one line of the alert log)

```
{"type":"Port Scan","severity":"medium","src":"127.0.0.1","dst":"127.0.0.1",
 "detail":"16 ports in 5 s","time":"17:57:50","latency_ms":2,"date":"2026-10-23"}
```

5.5 Reproducing the Tests

All of the tests can be reproduced on the development machine. The unit tests run with a single command and need no network, because they feed crafted packets directly to the detector. The integration tests need the server and sensor running, after which the two test scripts generate the attacks against the local host:

Running the automated tests

```
# unit tests (no network needed)
python -m unittest tests.test_rules -v

# integration test: with server and sensor running, generate attacks
python sensor/scan_test.py     # port scan
python sensor/flood_test.py    # SYN flood
```

The unit-test command reports ten tests and prints a line for each, ending with the word OK when all pass. Because the tests are fast and self-contained, they were run after every change to the detection code, which gave early warning whenever a change broke an existing rule.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 The Running System

The completed system runs as intended. Fig. 6.1 shows the dashboard under ordinary traffic: the three protocol counters update every second and the line chart tracks the total packet rate, while the alert table is empty. Fig. 6.2 shows the moment a port scan is detected: an orange banner names the attack and a medium-severity row is added to the table. Fig. 6.3 shows a SYN flood, which raises a red, high-severity alert. Fig. 6.4 shows the dashboard after the page has been reloaded; the alert history has been fetched from the server, so the earlier alerts are still present, and the per-type totals are shown above the table.

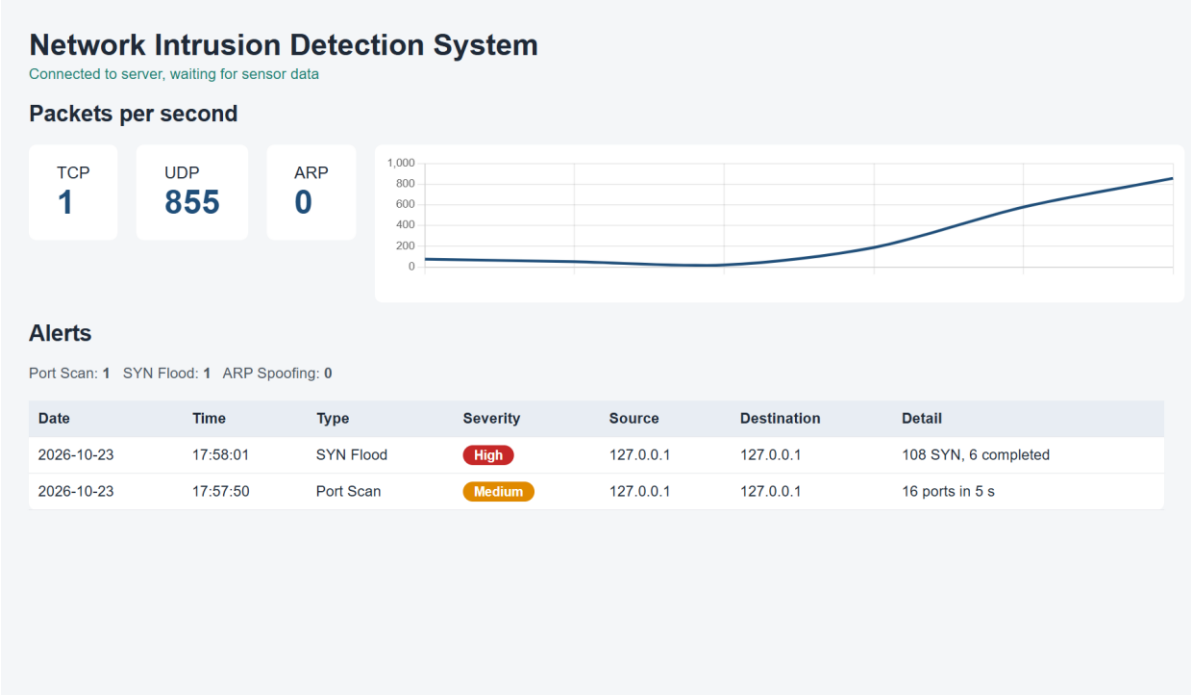


Fig. 6.1 Dashboard under normal traffic, with live counts and chart

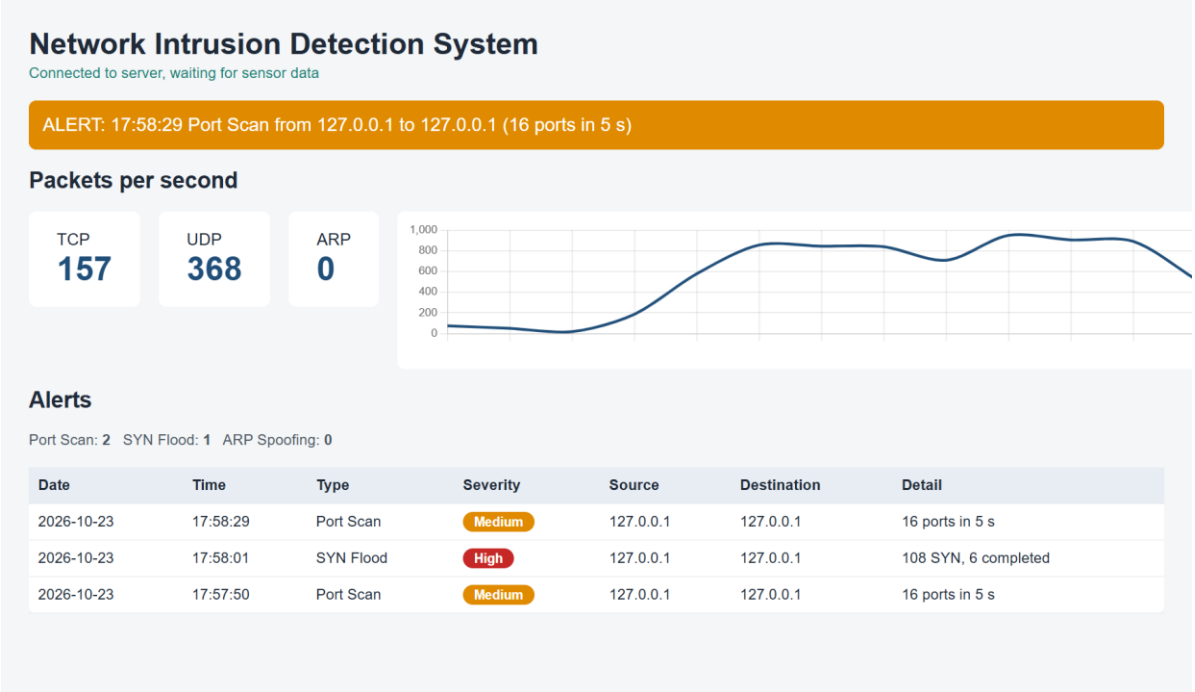


Fig. 6.2 Dashboard showing a detected port scan (medium severity)

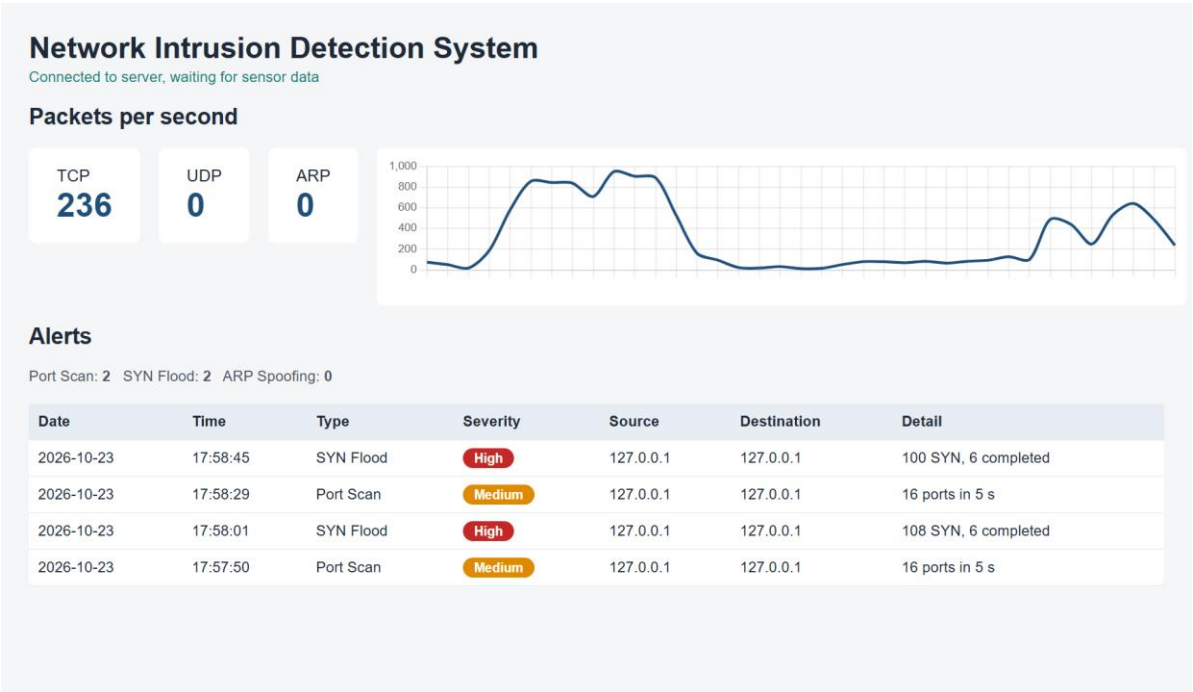


Fig. 6.3 Dashboard showing a detected SYN flood (high severity)

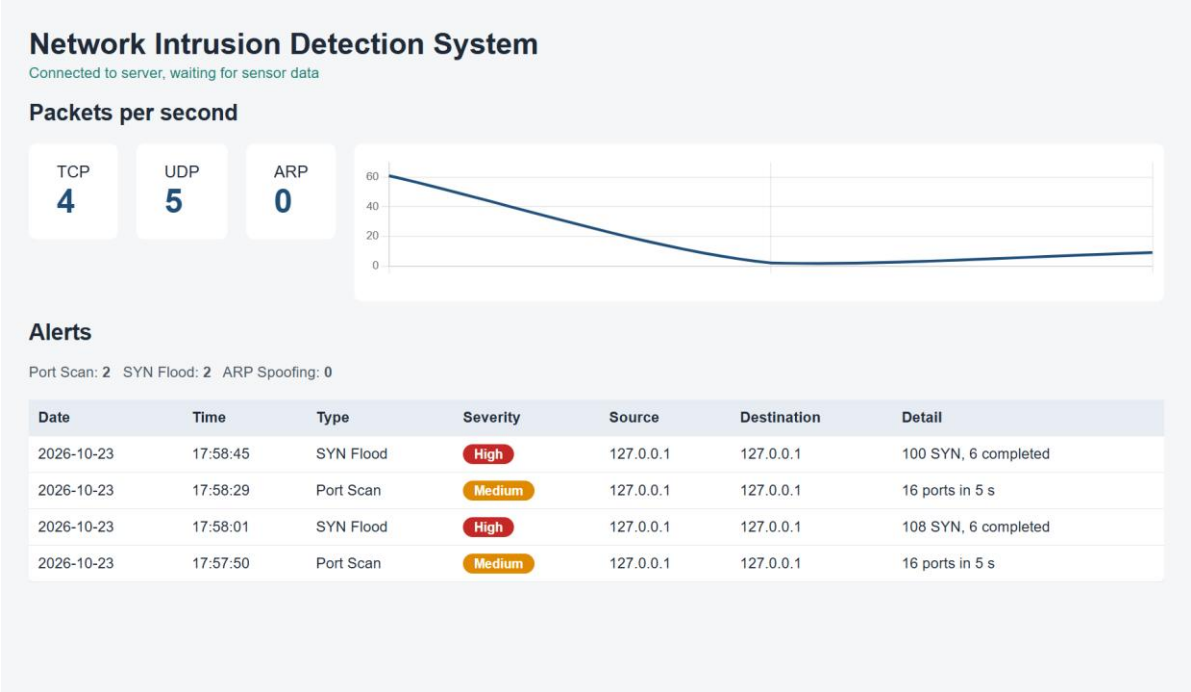


Fig. 6.4 Alert history preserved after a page reload

The behaviour of each attack is worth describing in a little more detail. For the port scan, the counters on the dashboard show a sharp rise in the TCP rate as the scan sweeps through the ports, and the alert appears at the moment the sixteenth distinct port is contacted; the detail field of the alert names the exact number of ports, which lets the administrator judge how aggressive the scan was. For the SYN flood, the TCP rate rises even more steeply because hundreds of SYN packets arrive in a short time, and the alert reports both the number of SYNs and the number of completed handshakes, so the imbalance that defines the attack is visible in the record itself. For ARP spoofing, which was exercised through the unit test, the alert is marked critical because a successful attack silently redirects all of the victim's traffic, which is more dangerous than the reconnaissance of a scan.

A useful property of the dashboard is that the chart gives context to the alerts. An administrator who sees a red banner can glance at the chart and confirm that the packet rate spiked at the same moment, which increases confidence that the alert is genuine rather than a stray event. This pairing of a specific alert with a live view of the overall traffic is exactly what the heavy, log-only tools discussed in Chapter 2 do not provide.

6.2 Performance Measurements

Table 6.1 summarises the measurements taken from the running system. The sensor handled several hundred packets per second on the loopback interface during the flood test without loss. The detection latency, measured as the time from the capture of the triggering packet to the creation of the alert, was of the order of two milliseconds in typical runs. One occasional outlier

of a few seconds was observed and traced to buffering in the Npcap loopback adapter rather than to the detector; it does not occur on a real interface. No false alarm was raised during ordinary web browsing over the observation period, which is the most important practical result: a detector that cried wolf would quickly be ignored.

Table 6.1 Performance measurements

Metric	Result
Lines of source code (sensor, server, dashboard)	about 200
Automated unit tests	10, all passing
Attack classes detected	3 (port scan, SYN flood, ARP spoofing)
Typical detection latency	about 2 ms
Packets handled during flood test	several hundred per second, no loss
False alarms during normal browsing	none observed

6.3 Discussion

The results confirm that a small, rule-based detector can recognise all three attacks in real time and present them clearly. Measured against the objectives of Chapter 1, every objective was met: live capture works, all three rules are implemented and tested, the alert pipeline and dashboard function end to end, and the system was verified by both unit and integration tests. The main strength of the design is transparency: every alert on the dashboard can be traced back to a counter and a threshold in Table 4.3, which is exactly what a machine-learning detector cannot easily offer. The design also keeps the capture path fast by doing the slow network work on a separate thread, and it fails safely when the server is unavailable.

The system has limitations, which are stated honestly. On a wireless network the sensor sees only broadcast traffic and traffic addressed to its own host, so full coverage of a switched network would need a mirror port or several sensors. A very slow scan, spread over a long period, or a scan spread across many source addresses, could stay under the thresholds; longer windows and per-destination counting would help. The ARP rule could not be demonstrated live on a single machine, although it is proven by the unit test. Finally, the server accepts an alert from any client on the same machine, so a production version should authenticate the sensor with a shared secret. These limitations point directly to the future work described in the next chapter, and none of them undermines the central result that the three attacks are detected correctly and shown within about a second.

6.4 Evaluation against the Objectives

Table 6.2 revisits the five objectives set out in Chapter 1 and records the outcome of each. All five were achieved within the scope defined for the project.

Table 6.2 Evaluation against the stated objectives

Objective	Outcome
Capture and decode live TCP, UDP and ARP packets	Achieved; verified against Wireshark
Rule-based detection of three attack classes	Achieved; all three rules implemented and tested
Alert pipeline to a Node.js and Express server	Achieved; alerts validated and logged
Real-time browser dashboard without refresh	Achieved; chart, banner and table update live
Verification and measurement	Achieved; 10 unit tests pass, latency measured

6.5 Summary of Strengths and Limitations

Table 6.3 places the strengths and the limitations of the system side by side, so that an honest overall picture is given. The strengths follow from the design goals of transparency and simplicity; the limitations are the natural price of a small, single-host prototype and are addressed by the future work.

Table 6.3 Strengths and limitations of the system

Strengths	Limitations
Every alert traceable to a counter and threshold	Sees only local and broadcast traffic on Wi-Fi
Real-time: alerts shown within about a second	Very slow or widely distributed scans may evade it
Small codebase, easy to read and extend	ARP rule not demonstrable live on one machine
Fails safely when the server is unavailable	Server does not yet authenticate the sensor
No training data or heavy configuration needed	Only three attack classes are covered so far

6.6 Comparison with Existing Tools in Practice

It is worth restating, in the light of the working system, how the project compares with the tools reviewed in Chapter 2. Wireshark would have captured the same packets, but an operator would have had to watch the capture and notice the scan by eye; the project automates exactly that judgement and raises the alert itself. Snort could have detected the same attacks, but it would have required a rule file in its own syntax and would have written to a log rather than to a live screen. The project's dashboard shows the alert and the traffic together, in a browser, with no

configuration language to learn. Against a machine-learning detector, the project cannot claim higher accuracy on a benchmark, but it can explain every decision it makes, which for a small network and for a viva is the more useful property. The comparison therefore confirms the gap identified in Chapter 2 and shows that the delivered system fills it.

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project set out to build a lightweight, real-time network intrusion detection system that a small network could actually use and that a student could explain in full. All of the objectives stated in Chapter 1 were met. The system captures live TCP, UDP and ARP traffic, detects port scans, SYN floods and ARP spoofing with transparent rules, sends each alert through a Node.js server, and displays live counts and colour-coded alerts on a web dashboard within about a second. It was verified by ten automated unit tests, all passing, and by self-generated attack traffic, and its detection latency was measured in milliseconds. The whole system is about two hundred lines of code, so every line can be understood and defended. Its value lies not in a new algorithm but in the combination of explainable detection, a simple alert pipeline and a real-time dashboard in a package small enough to teach from.

Beyond the working software, the project was a thorough exercise in computer networking and security. Building the rules required a precise understanding of the TCP three-way handshake, of the stateless nature of the Address Resolution Protocol, and of how packet capture works on a modern operating system. The discipline of writing a unit test for every rule, including the negative cases, gave a practical appreciation of why professional detectors are judged as much by their false-alarm rate as by what they catch.

The project also reinforced several good engineering habits. Keeping the thresholds in a configuration file rather than scattering them through the code made the system easy to tune and to demonstrate. Separating the fast capture path from the slower network path with a queue and a dedicated thread kept the detector responsive under load. Writing the dashboard to insert network text safely, and the server to reject malformed alerts, showed that even a small system benefits from thinking about how it could be misused. These habits are as valuable a takeaway as the finished detector itself, and they will carry over to larger projects in later semesters.

Finally, the choice to build the system from first principles rather than to assemble it from a large existing framework proved to be the right one for the goals set out at the start. It meant that nothing in the system is a black box: every packet that is captured, every counter that is incremented, and every alert that is raised can be followed through the code and explained. For a learning project that must be defended in person, that transparency is worth more than the extra features a heavier framework would have provided.

7.2 Future Scope

The design was deliberately kept open so that it can grow. The following extensions are planned:

- Additional rules for other attacks, such as ICMP floods, DNS tunnelling and SSH brute-force attempts, each added as a small, independent function in the same style as the three existing rules.
- Notification by e-mail or messaging so that an administrator is told of an alert even when the dashboard is not open.
- Storage of alerts in a lightweight database such as SQLite, so that past alerts can be searched and filtered by type, severity, address or date.
- Replay of saved capture files, so that recorded traffic can be analysed offline and so that the ARP rule can be demonstrated from a recorded attack.
- Authentication of the sensor to the server with a shared secret, so that only the genuine sensor can submit alerts.
- An optional active-response mode that adds a firewall rule against a confirmed attacker, turning the detection system into a prevention system.

With these additions the system could move from a teaching prototype towards a genuinely useful monitor for a small network, while keeping the transparency that is its central design principle. The modular rule engine means that each extension can be added, tested and explained on its own, without disturbing the parts that already work.

REFERENCES

The references below are listed in the order in which they are first cited in the report.

[1] W. Stallings, Network Security Essentials: Applications and Standards, 6th ed. Harlow, England: Pearson, 2017.

[2] K. Scarfone and P. Mell, “Guide to Intrusion Detection and Prevention Systems (IDPS),” National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Special Publication 800-94, 2007.

[3] D. E. Denning, “An Intrusion-Detection Model,” IEEE Transactions on Software Engineering, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.

[4] M. Roesch, “Snort – Lightweight Intrusion Detection for Networks,” in Proc. 13th USENIX Systems Administration Conference (LISA ’99), Seattle, WA, USA, 1999, pp. 229–238.

[5] V. Paxson, “Bro: A System for Detecting Network Intruders in Real-Time,” Computer Networks, vol. 31, no. 23–24, pp. 2435–2463, Dec. 1999.

[6] J. Jung, V. Paxson, A. W. Berger, and H. Balakrishnan, “Fast Portscan Detection Using Sequential Hypothesis Testing,” in Proc. IEEE Symposium on Security and Privacy, Oakland, CA, USA, 2004, pp. 211–225.

[7] H. Wang, D. Zhang, and K. G. Shin, “Detecting SYN Flooding Attacks,” in Proc. IEEE INFOCOM, New York, NY, USA, 2002, vol. 3, pp. 1530–1539.

[8] W. Eddy, “TCP SYN Flooding Attacks and Common Mitigations,” Internet Engineering Task Force, RFC 4987, Aug. 2007.

[9] D. C. Plummer, “An Ethernet Address Resolution Protocol,” Internet Engineering Task Force, RFC 826, Nov. 1982.

[10] V. Ramachandran and S. Nandi, “Detecting ARP Spoofing: An Active Technique,” in Proc. International Conference on Information Systems Security (ICISS), Kolkata, India, 2005, Lecture Notes in Computer Science, vol. 3803, pp. 239–250.

[11] R. Sommer and V. Paxson, “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection,” in Proc. IEEE Symposium on Security and Privacy, Berkeley, CA, USA, 2010, pp. 305–316.

[12] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, “A Detailed Analysis of the KDD CUP 99 Data Set,” in Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), Ottawa, ON, Canada, 2009, pp. 1–6.

[13] P. Biondi and the Scapy community, “Scapy Documentation, Release 2.6,” 2024. [Online]. Available: https://scapy.readthedocs.io. [Accessed: 23-Oct-2026].

[14] Nmap Project, “Npcap: Packet Capture Library for Windows,” 2024. [Online]. Available: https://npcap.com. [Accessed: 23-Oct-2026].

[15] OpenJS Foundation, “Node.js Documentation,” 2025. [Online]. Available: https://nodejs.org/docs. [Accessed: 23-Oct-2026].

[16] OpenJS Foundation, “Express 5.x API Reference,” 2025. [Online]. Available: https://expressjs.com/en/5x/api.html. [Accessed: 23-Oct-2026].

[17] Socket.IO, “Socket.IO Documentation, Version 4,” 2025. [Online]. Available: https://socket.io/docs/v4. [Accessed: 23-Oct-2026].

[18] Wireshark Foundation, “Wireshark User’s Guide,” 2025. [Online]. Available: https://www.wireshark.org/docs. [Accessed: 23-Oct-2026].

[19] Python Software Foundation, “The Python Standard Library, Release 3.13,” 2025. [Online]. Available: https://docs.python.org/3/library/. [Accessed: 23-Oct-2026].

[20] R. Fielding and J. Reschke, “Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing,” Internet Engineering Task Force, RFC 7230, Jun. 2014.

[21] T. Bray, Ed., “The JavaScript Object Notation (JSON) Data Interchange Format,” Internet Engineering Task Force, RFC 8259, Dec. 2017.

[22] Chart.js, “Chart.js Documentation, Version 4,” 2025. [Online]. Available: https://www.chartjs.org/docs/latest/. [Accessed: 23-Oct-2026].

[23] OWASP Foundation, “Cross Site Scripting Prevention Cheat Sheet,” OWASP Cheat Sheet Series, 2025. [Online]. Available: https://cheatsheetseries.owasp.org. [Accessed: 23-Oct-2026].

APPENDIX A: SOURCE CODE

The complete source code of the project is listed below. It is also available in the project repository. The sensor is written in Python, the server in Node.js, and the dashboard in HTML, CSS and JavaScript.

A.1 sensor/config.json

```
{
  "interface": "Wi-Fi",
  "server": "http://localhost:3000",
  "gateway": "192.168.29.1",
  "gateway_mac": "",
  "window_seconds": 5,
  "port_scan_ports": 15,
  "syn_flood_syns": 100,
  "cooldown_seconds": 30
}
```

A.2 sensor/nids.py

```
import json
import os
import queue
import threading
import time
import urllib.request
from collections import defaultdict, deque

from scapy.all import ARP, IP, TCP, UDP, conf, getmacbyip, sniff

HERE = os.path.dirname(os.path.abspath(__file__))
cfg = json.load(open(os.path.join(HERE, "config.json")))
WINDOW = cfg["window_seconds"]
COOLDOWN = cfg["cooldown_seconds"]

counts = {"TCP": 0, "UDP": 0, "ARP": 0}
syn_by_src = defaultdict(deque)
syn_by_dst = defaultdict(deque)
ack_by_dst = defaultdict(deque)
trusted = {}
last_alert = {}
outbox = queue.Queue()
live = True


def send(path, data):
    request = urllib.request.Request(cfg["server"] + path, json.dumps(data).encode(),
{"Content-Type": "application/json"})
    try:
        urllib.request.urlopen(request, timeout=2)
    except OSError:
        print("server not reachable")


def sender():
    while True:
        path, data = outbox.get()
        send(path, data)
        outbox.task_done()
```

```
def stats():
    while True:
        time.sleep(1)
        snapshot = dict(counts)
        for name in counts:
            counts[name] = 0
        print(snapshot)
        outbox.put(("/stats", snapshot))


def trim(history, now):
    while history and now - history[0][0] > WINDOW:
        history.popleft()


def alert(kind, severity, src, dst, detail, packet, subject):
    now = float(packet.time)
    if now - last_alert.get((kind, subject), 0) < COOLDOWN:
        return
    last_alert[(kind, subject)] = now
    record = {"type": kind, "severity": severity, "src": src, "dst": dst, "detail":
detail,
              "time": time.strftime("%H:%M:%S", time.localtime(now))}
    if live:
        record["latency_ms"] = round((time.time() - now) * 1000)
    print("ALERT", record)
    outbox.put(("/alert", record))


def check_tcp(packet):
    ip, tcp = packet[IP], packet[TCP]
    now = float(packet.time)
    if tcp.flags == "S":
        history = syn_by_src[ip.src]
        history.append((now, tcp.dport))
        trim(history, now)
        ports = {port for _, port in history}
        if len(ports) > cfg["port_scan_ports"]:
            alert("Port Scan", "medium", ip.src, ip.dst, "%d ports in %d s" %
(len(ports), WINDOW), packet, ip.src)
        syns = syn_by_dst[ip.dst]
        syns.append((now, tcp.dport))
        trim(syns, now)
        trim(ack_by_dst[ip.dst], now)
        completed = len(ack_by_dst[ip.dst])
        target_ports = {port for _, port in syns}
        if len(syns) >= cfg["syn_flood_syns"] and len(target_ports) <= 3 and completed <
len(syns) / 10:
            alert("SYN Flood", "high", ip.src, ip.dst, "%d SYN, %d completed" %
(len(syns), completed), packet, ip.dst)
    elif "A" in tcp.flags and "R" not in tcp.flags:
        ack_by_dst[ip.dst].append((now, tcp.dport))


def check_arp(packet):
    arp = packet[ARP]
    known = trusted.get(arp.psrc)
    if arp.op == 2 and known and arp.hwsrc.lower() != known.lower():
        alert("ARP Spoofing", "critical", arp.psrc, arp.pdst, "MAC changed from %s to
%s" % (known, arp.hwsrc), packet, arp.psrc)


def handle(packet):
    for name, layer in (("TCP", TCP), ("UDP", UDP), ("ARP", ARP)):
        if packet.haslayer(layer):
            counts[name] += 1
    if packet.haslayer(IP) and packet.haslayer(TCP):
```

```
        check_tcp(packet)
    elif packet.haslayer(ARP):
        check_arp(packet)


def learn_gateway():
    mac = cfg["gateway_mac"] or getmacbyip(cfg["gateway"])
    if mac:
        trusted[cfg["gateway"]] = mac
        print("trusted gateway", cfg["gateway"], "at", mac)
    else:
        print("gateway MAC unknown, ARP rule is off")


if __name__ == "__main__":
    learn_gateway()
    threading.Thread(target=sender, daemon=True).start()
    threading.Thread(target=stats, daemon=True).start()
    print("NIDS sensor started on", cfg["interface"], "press Ctrl+C to stop")
    sniff(iface=[cfg["interface"], conf.loopback_name], filter="(tcp or udp or arp) and
not port 3000", prn=handle, store=False)
```

A.3 sensor/scan_test.py

```
import socket

for port in range(1, 101):
    s = socket.socket()
    s.settimeout(0.05)
    s.connect_ex(("127.0.0.1", port))
    s.close()

print("knocked on 100 ports of 127.0.0.1")
```

A.4 sensor/flood_test.py

```
import socket

for _ in range(300):
    s = socket.socket()
    s.settimeout(0.02)
    s.connect_ex(("127.0.0.1", 9))
    s.close()

print("sent 300 connection attempts to port 9 of 127.0.0.1")
```

A.5 server/server.js

```
const express = require("express");
const http = require("http");
const fs = require("fs");
const { Server } = require("socket.io");

const app = express();
const server = http.createServer(app);
const io = new Server(server);
const LOG = __dirname + "/alerts.log";

app.use(express.json());
app.use(express.static(__dirname + "/public"));

app.post("/stats", (req, res) => {
  io.emit("stats", req.body);
  res.sendStatus(200);
});
```

```
app.post("/alert", (req, res) => {
  const alert = req.body;
  if (!alert.type || !alert.src || !alert.severity) return res.sendStatus(400);
  alert.date = new Date().toISOString().slice(0, 10);
  fs.appendFileSync(LOG, JSON.stringify(alert) + "\n");
  io.emit("alert", alert);
  res.sendStatus(200);
});

app.get("/api/alerts", (req, res) => {
  const lines = fs.existsSync(LOG) ? fs.readFileSync(LOG,
"utf8").trim().split("\n").filter(Boolean) : [];
  res.json(lines.slice(-100).map(line => JSON.parse(line)));
});

server.listen(3000, () => console.log("Dashboard running at http://localhost:3000"));
```

A.6 server/public/index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>NIDS Dashboard</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 30px; background: #f4f6f8; color:
#1f2937; }
    h1 { margin: 0 0 4px 0; }
    #status { color: #6b7280; margin-bottom: 20px; }
    #status.on { color: #1b7f73; }
    .row { display: flex; gap: 20px; align-items: flex-start; }
    .stats { display: flex; gap: 20px; }
    .stats div { background: white; padding: 20px 30px; border-radius: 8px; font-size:
18px; }
    .stats span { display: block; font-size: 36px; font-weight: bold; color: #1f4e79; }
    .chart { background: white; border-radius: 8px; padding: 10px; flex: 1; height:
150px; }
    #banner { color: white; padding: 15px; font-size: 20px; border-radius: 8px; margin-
bottom: 20px; }
    .medium { background: #e08a00; }
    .high { background: #c62828; }
    .critical { background: #7b1fa2; }
    table { width: 100%; border-collapse: collapse; background: white; border-radius:
8px; }
    th, td { text-align: left; padding: 10px; border-bottom: 1px solid #e5e7eb; }
    th { background: #e8edf3; }
    .pill { color: white; font-weight: bold; text-transform: capitalize; padding: 3px
12px; border-radius: 12px; font-size: 14px; }
    .pill.medium { background: #e08a00; } .pill.high { background: #c62828; }
.pill.critical { background: #7b1fa2; }
    .totals { margin: 10px 0 20px 0; color: #4b5563; }
  </style>
</head>
<body>
  <h1>Network Intrusion Detection System</h1>
  <div id="status">Connecting to server...</div>
  <div id="banner" hidden></div>

  <h2>Packets per second</h2>
  <div class="row">
    <div class="stats">
      <div>TCP <span id="TCP">0</span></div>
      <div>UDP <span id="UDP">0</span></div>
      <div>ARP <span id="ARP">0</span></div>
    </div>
    <div class="chart"><canvas id="chart"></canvas></div>
  </div>
```

```
  <h2>Alerts</h2>
  <div class="totals">Port Scan: <b id="total-Port-Scan">0</b>   SYN Flood: <b
id="total-SYN-Flood">0</b>   ARP Spoofing: <b id="total-ARP-Spoofing">0</b></div>
  <table>

<thead><tr><th>Date</th><th>Time</th><th>Type</th><th>Severity</th><th>Source</th><th>De
stination</th><th>Detail</th></tr></thead>
    <tbody id="alerts"></tbody>
  </table>

  <script src="/socket.io/socket.io.js"></script>
  <script src="/chart.umd.js"></script>
  <script>
    const socket = io();
    const status = document.getElementById("status");
    socket.on("connect", () => { status.textContent = "Connected to server, waiting for
sensor data"; status.className = "on"; });
    socket.on("disconnect", () => { status.textContent = "Server connection lost";
status.className = ""; });

    const chart = new Chart(document.getElementById("chart"), {
      type: "line",
      data: { labels: [], datasets: [{ label: "packets per second", data: [],
borderColor: "#1f4e79", tension: 0.3, pointRadius: 0 }] },
      options: { animation: false, maintainAspectRatio: false, scales: { y: {
beginAtZero: true } }, plugins: { legend: { display: false } } }
    });

    socket.on("stats", data => {
      for (const name in data) document.getElementById(name).textContent = data[name];
      chart.data.labels.push("");
      chart.data.datasets[0].data.push(data.TCP + data.UDP + data.ARP);
      if (chart.data.labels.length > 60) { chart.data.labels.shift();
chart.data.datasets[0].data.shift(); }
      chart.update();
    });

    const totals = {};
    function addRow(alert, top) {
      const row = document.createElement("tr");
      for (const value of [alert.date, alert.time, alert.type, alert.severity,
alert.src, alert.dst, alert.detail]) {
        const cell = document.createElement("td");
        if (value === alert.severity) {
          const pill = document.createElement("span");
          pill.className = "pill " + alert.severity;
          pill.textContent = value;
          cell.appendChild(pill);
        } else {
          cell.textContent = value;
        }
        row.appendChild(cell);
      }
      const body = document.getElementById("alerts");
      top ? body.prepend(row) : body.appendChild(row);
      totals[alert.type] = (totals[alert.type] || 0) + 1;
      const box = document.getElementById("total-" + alert.type.replace(" ", "-"));
      if (box) box.textContent = totals[alert.type];
    }

    fetch("/api/alerts").then(r => r.json()).then(list => list.reverse().forEach(a =>
addRow(a, false)));

    socket.on("alert", alert => {
      addRow(alert, true);
      const banner = document.getElementById("banner");
      banner.textContent = "ALERT: " + alert.time + " " + alert.type + " from " +
alert.src + " to " + alert.dst + " (" + alert.detail + ")";
```

```
      banner.className = alert.severity;
      banner.hidden = false;
      setTimeout(() => banner.hidden = true, 6000);
    });
  </script>
</body>
</html>
```

A.7 tests/test_rules.py

```
import os
import queue
import sys
import unittest

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)), "..",
"sensor"))
import nids
from scapy.all import ARP, Ether, IP, TCP

GATEWAY = nids.cfg["gateway"]
GOOD_MAC = "aa:bb:cc:dd:ee:01"
BAD_MAC = "de:ad:be:ef:00:01"
VICTIM = "192.168.29.171"


def tcp(src, dst, dport, flags, t, sport=40000):
    p = Ether() / IP(src=src, dst=dst) / TCP(sport=sport, dport=dport, flags=flags)
    p.time = t
    return p


def arp(op, ip, mac, t):
    p = Ether(src=mac) / ARP(op=op, psrc=ip, hwsrc=mac, pdst=VICTIM)
    p.time = t
    return p


class RuleTests(unittest.TestCase):
    def setUp(self):
        for store in (nids.syn_by_src, nids.syn_by_dst, nids.ack_by_dst,
nids.last_alert, nids.trusted):
            store.clear()
        nids.trusted[GATEWAY] = GOOD_MAC
        nids.outbox = queue.Queue()
        nids.live = False

    def alerts(self):
        found = []
        while not nids.outbox.empty():
            path, data = nids.outbox.get()
            if path == "/alert":
                found.append(data)
        return found

    def test_port_scan_fires_once(self):
        for port in range(1, 21):
            nids.handle(tcp("10.0.0.5", VICTIM, port, "S", 100 + port * 0.01))
        found = self.alerts()
        self.assertEqual(len(found), 1)
        self.assertEqual(found[0]["type"], "Port Scan")
        self.assertEqual(found[0]["severity"], "medium")
        self.assertEqual(found[0]["src"], "10.0.0.5")

    def test_few_ports_no_alert(self):
        for port in range(1, 11):
```

```
            nids.handle(tcp("10.0.0.5", VICTIM, port, "S", 100 + port * 0.01))
        self.assertEqual(self.alerts(), [])

    def test_syn_ack_replies_are_ignored(self):
        for port in range(1, 31):
            nids.handle(tcp(VICTIM, "10.0.0.5", port, "SA", 100 + port * 0.01))
        self.assertEqual(self.alerts(), [])

    def test_slow_scan_stays_under_window(self):
        for port in range(1, 31):
            nids.handle(tcp("10.0.0.5", VICTIM, port, "S", 100 + port * 1.0))
        self.assertEqual(self.alerts(), [])

    def test_cooldown_blocks_repeat_alert(self):
        for port in range(1, 21):
            nids.handle(tcp("10.0.0.5", VICTIM, port, "S", 100 + port * 0.01))
        for port in range(100, 140):
            nids.handle(tcp("10.0.0.5", VICTIM, port, "S", 110 + port * 0.01))
        self.assertEqual(len(self.alerts()), 1)
        for port in range(200, 240):
            nids.handle(tcp("10.0.0.5", VICTIM, port, "S", 150 + port * 0.01))
        self.assertEqual(len(self.alerts()), 1)

    def test_syn_flood_fires(self):
        for i in range(120):
            nids.handle(tcp("10.0.%d.%d" % (i // 250, i % 250 + 1), VICTIM, 80, "S", 200
+ i * 0.01, sport=1000 + i))
        found = self.alerts()
        self.assertEqual(len(found), 1)
        self.assertEqual(found[0]["type"], "SYN Flood")
        self.assertEqual(found[0]["severity"], "high")

    def test_completed_handshakes_are_not_a_flood(self):
        for i in range(120):
            t = 200 + i * 0.01
            nids.handle(tcp("10.0.0.5", VICTIM, 80, "S", t, sport=1000 + i))
            nids.handle(tcp(VICTIM, "10.0.0.5", 1000 + i, "SA", t + 0.001, sport=80))
            nids.handle(tcp("10.0.0.5", VICTIM, 80, "A", t + 0.002, sport=1000 + i))
        self.assertEqual(self.alerts(), [])

    def test_port_scan_is_not_reported_as_flood(self):
        for port in range(1, 121):
            nids.handle(tcp("10.0.0.5", VICTIM, port, "S", 300 + port * 0.01))
        found = self.alerts()
        self.assertEqual([a["type"] for a in found], ["Port Scan"])

    def test_arp_spoof_fires(self):
        nids.handle(arp(2, GATEWAY, BAD_MAC, 400))
        found = self.alerts()
        self.assertEqual(len(found), 1)
        self.assertEqual(found[0]["type"], "ARP Spoofing")
        self.assertEqual(found[0]["severity"], "critical")
        self.assertEqual(found[0]["src"], GATEWAY)

    def test_genuine_arp_traffic_is_ignored(self):
        nids.handle(arp(2, GATEWAY, GOOD_MAC, 400))
        nids.handle(arp(1, GATEWAY, BAD_MAC, 401))
        nids.handle(arp(2, "192.168.29.77", BAD_MAC, 402))
        self.assertEqual(self.alerts(), [])


if __name__ == "__main__":
    unittest.main(verbosity=2)
```